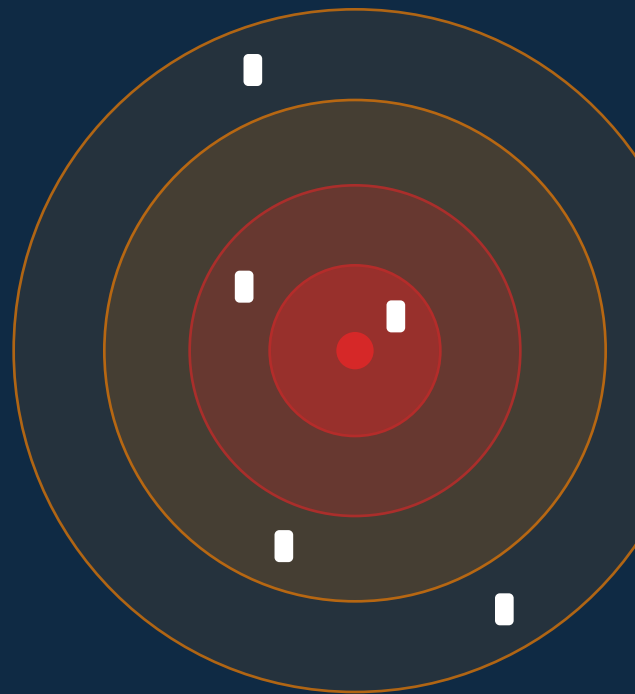


ALERTA CERCA

Sistema Inteligente de Alertamiento Comunitario por Proximidad

Propuesta de solución y guía de implementación paso a paso
para un equipo de 5 integrantes



"Si algo importante está ocurriendo cerca de ti, deberías poder saberlo."

Concepto rector del reto

Equipo

Escriban su nombre junto al rol que cada quien tomará (ver sección 7).

R1 Base de datos y backend geográfico (líder técnico)

R2 Servicios en la nube y notificaciones

R3 App móvil: pantallas y experiencia

R4 App móvil: ubicación, notificaciones y sistema

R5 Panel de validadores, producto y presentación

Contenido

1 Resumen ejecutivo	4
2 Entendiendo el reto	5
2.1 El problema en una frase	5
2.2 Qué pide exactamente el CCE	5
2.3 El flujo mínimo que debemos demostrar (sección 12 del reto)	5
2.4 Lo que el reto deja claro	5
2.5 Lo que el reto privilegia y cómo respondemos	6
3 La solución propuesta	7
3.1 Arquitectura general	7
3.2 Las cuatro piezas	7
3.3 Tecnologías y por qué	8
4 Diseño detallado: lo que nos diferencia	9
4.1 Categorías y niveles	9
4.2 Radio dinámico en anillos	9
4.3 Privacidad por diseño: celdas geohash	10
4.4 Estados de una alerta (confiabilidad)	11
4.5 Contra alertas falsas, maliciosas, duplicadas o viejas	12
4.6 El riesgo que pocos equipos van a ver	12
4.7 Cómo se ven las notificaciones	13
5 ¿Se puede alertar a un celular sin app?	14
5.1 Estrategia de adopción: una app solo sirve si la tienen	14
6 Cómo cumplimos cada punto del reto	16
6.1 Restricciones (sección 13 del reto)	16
7 Organización del equipo	17
7.1 Roles y responsabilidades	17
7.2 Contratos entre roles: acuérdense antes de programar	17
7.3 Estructura del repositorio	18
7.4 Reglas de trabajo	18
8 Plan de implementación paso a paso	19

8.1 Cronograma por rol	19
8.2 Prioridades: qué hacer primero si falta tiempo	19
8.3 Fase 0 · Preparación (antes del hackatón)	19
8.4 Fase 1 · Cimientos (horas 0 a 6)	22
8.5 Fase 2 · Flujo principal (horas 6 a 16)	29
8.6 Fase 3 · Inteligencia y confianza (horas 16 a 26)	37
8.7 Fase 4 · Canales alternos e integración (horas 26 a 34)	42
8.8 Fase 5 · Pruebas, datos de la demo y ensayo (horas 34 a 40)	45
9 Guion de la demostración y del pitch	48
9.1 La historia para abrir (45 segundos)	48
9.2 Minuto a minuto (7 minutos)	48
9.3 Preguntas difíciles del jurado	48
10 Riesgos y plan B	49
11 Escalamiento y siguientes pasos	49
11.1 Del prototipo al piloto	49
11.2 Mejoras técnicas	49
11.3 Costos estimados	50
12 Privacidad y marco legal	51
A Glosario para el equipo	52
B Ejemplo de una alerta en formato CAP	52
C Lista de verificación para el día de la demo	53

1 Resumen ejecutivo

ALERTA CERCA es una red de alertas comunitarias que hace llegar avisos importantes (un menor desaparecido, un incendio, un vehículo robado) **primero a las personas que están cerca del lugar**, y amplía la zona de difusión conforme pasa el tiempo. Funciona con una app móvil gratuita, un panel web para quienes validan la información y canales alternos para quien no quiera instalar nada.

\$0

costo del prototipo
(planes gratuitos)

~1 km

lo máximo que el servidor
sabe de tu ubicación

4

niveles de alerta y
4 estados de confianza

5

roles trabajando
en paralelo

La propuesta en 6 decisiones

1. **App multiplataforma en Flutter:** un solo código para Android e iOS (objetivo específico 1 del reto).
2. **Backend en Supabase (PostgreSQL + PostGIS):** base de datos con consultas geográficas en SQL, cuentas de usuario, fotos y funciones en la nube en un solo servicio con plan gratuito.
3. **Notificaciones con Firebase Cloud Messaging (FCM):** gratis e independientes de la compañía telefónica; solo requieren internet (datos o wifi).
4. **Privacidad por diseño:** el servidor solo conoce la celda de ~1 km donde está el teléfono, nunca su ubicación exacta ni su historial de recorridos.
5. **Confianza por niveles:** los reportes ciudadanos se marcan como "no confirmados" y viajan poco; las alertas verificadas por una institución viajan lejos. Los casos de personas y menores siempre se validan antes.
6. **Más allá de la app:** bot de Telegram para quien no quiera instalar nada, y salida en formato CAP (estándar internacional) para que Protección Civil pueda retransmitir las alertas verificadas.

Lo que vamos a demostrar

Cuatro teléfonos en puntos distintos. Un ciudadano reporta un menor desaparecido; queda en revisión. Un validador lo verifica desde el panel. El teléfono a 300 m suena al instante, el de 2.6 km recibe la alerta cuando el radio crece a 3 km, el de 6 km cuando crece a 10 km, y el de Zihuatanejo (76 km) nunca la recibe. Al final se marca como resuelta y todos ven que el caso terminó.

La idea central

El reto no pide "una app": pide que **la información correcta llegue a las personas correctas, en el lugar correcto y en el momento útil**. La app es el medio; lo que nos diferencia es el radio dinámico, la privacidad, la confianza y el análisis honesto de cómo llegar a quien no tiene la app.

2 Entendiendo el reto

2.1 El problema en una frase

Hoy la información de emergencias **se publica y se espera que alguien la vea** (Facebook, grupos de WhatsApp, páginas de noticias). El reto del Consejo Coordinador Empresarial (CCE) de Lázaro Cárdenas pide pasar a un modelo donde **la información relevante llegue a quien está cerca, cuando todavía sirve**. Una persona puede estar a 200 metros de donde desapareció un niño y no enterarse, mientras la publicación la ven personas a cientos de kilómetros.

2.2 Qué pide exactamente el CCE

- Un **prototipo funcional (MVP)** llamado **ALERTA CERCA**, gratuito para el usuario e independiente de la compañía telefónica.
- Alertas **asociadas a una ubicación** que lleguen a quienes estén dentro de un **radio** (1, 3, 5, 10 km o más).
- Un **radio dinámico** que se amplíe conforme pasa el tiempo o cambia la situación.
- **Categorías y niveles**: menores, personas desaparecidas, adultos mayores, robo de vehículos, asaltos, incendios, inundaciones, accidentes, riesgos ambientales, evacuaciones, fenómenos naturales y otros.
- Distinguir **alertas verificadas** de **reportes ciudadanos**, evitar alertas falsas, duplicadas o viejas, y poder **cerrarlas** cuando se resuelvan.
- **Privacidad**, seguridad y posibilidad de **integrarse** después con autoridades y otras instituciones.

2.3 El flujo mínimo que debemos demostrar (sección 12 del reto)

#	Paso del reto	Qué pasa en ALERTA CERCA	Dónde vive
1	Ocurre un acontecimiento	Un ciudadano o una institución lo presencia.	—
2	Se genera una alerta	Reporte en 3 pasos desde la app o el panel.	RPC crear_reporte
3	Se geolocaliza	GPS del teléfono o pin en el mapa; se guarda como punto geográfico.	alertas.ubicacion
4	Nivel de importancia y radio	La categoría define el nivel y los escalones de radio; el estado de verificación pone un tope.	categorias, radio_permitido()
5	Se identifican usuarios cercanos	Consulta espacial sobre las celdas de ~1 km de los teléfonos y sus zonas guardadas.	dispositivos_objetivo()
6	Se distribuye la información	Una función en la nube envía push (FCM) y mensajes de Telegram.	Edge Function notificar
7	La comunidad recibe la alerta	Notificación con distancia, nivel, estado y acciones.	App / Telegram

2.4 Lo que el reto deja claro

- **No sustituye** al 911, a la Alerta Amber, al alertamiento sísmico ni a otros sistemas oficiales: es un mecanismo **complementario**. Hay que decirlo en la app y en la presentación.
- Es un prototipo en **entorno controlado**: no hace falta publicar la app en las tiendas.
- **No imponen tecnología**: nosotros elegimos y tenemos que **justificar** cada decisión.

- La sección 13 pide investigar las **limitaciones para alertar a un celular sin app instalada**. No es un impedimento: es parte del problema y le dedicamos la sección 5 completa.

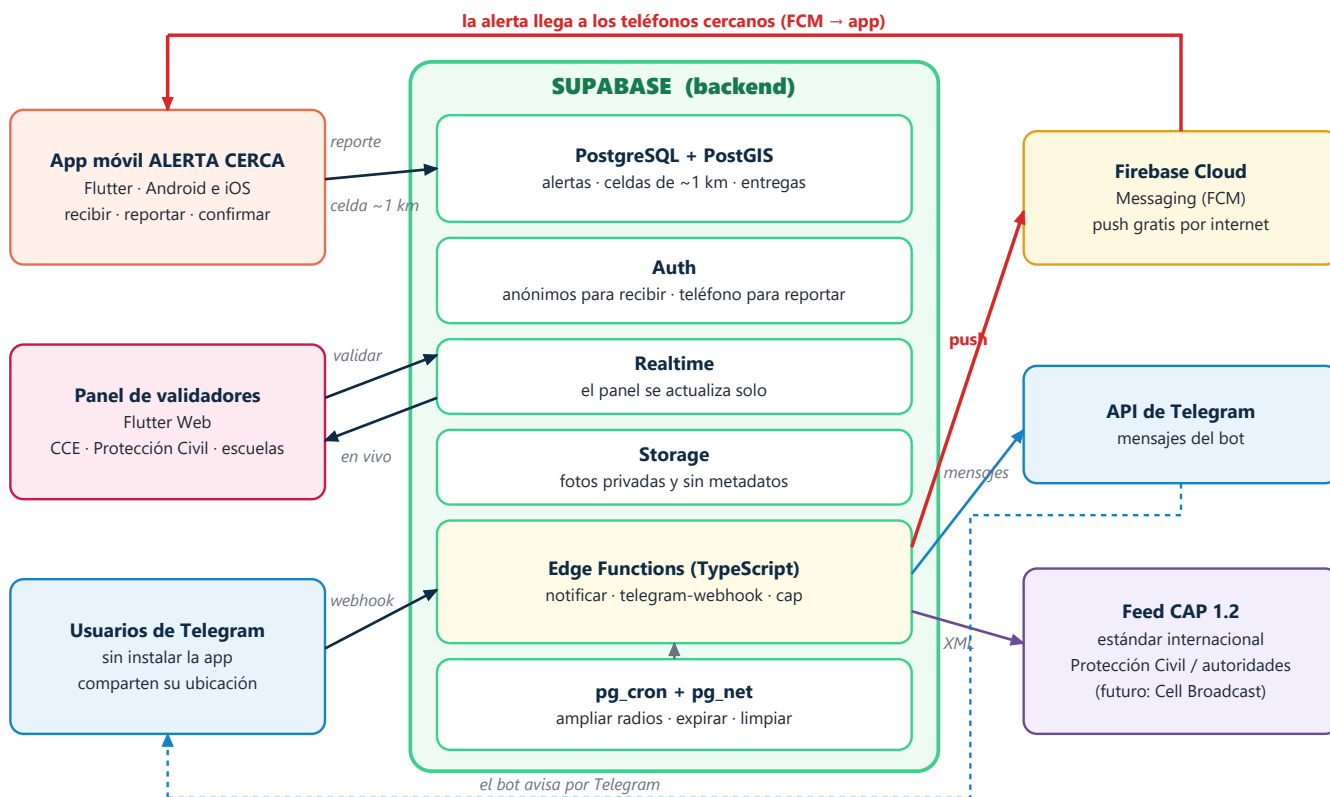
2.5 Lo que el reto privilegia y cómo respondemos

Criterio	Cómo lo atiende ALERTA CERCA
Rapidez	Push de alta prioridad; consulta geográfica con índice espacial (milisegundos); una alerta verificada sale en segundos.
Facilidad de uso	Recibir alertas no requiere crear cuenta; reportar toma 3 pasos; la notificación dice qué pasó, a qué distancia y qué hacer.
Confiabilidad	Estados visibles (en revisión, no confirmada, corroborada, verificada); validadores institucionales; confirmaciones de vecinos.
Seguridad	Reglas de acceso por fila (RLS) en cada tabla; las operaciones sensibles solo pasan por funciones del servidor; cuentas verificadas para reportar.
Bajo costo	Todo en planes gratuitos; mapas de OpenStreetMap sin llave de pago.
Escalabilidad	PostgreSQL + PostGIS con índices espaciales; FCM maneja millones de mensajes; diseño listo para varias ciudades.
Protección de datos	Celdas de ~1 km, fotos sin metadatos, borrado automático a 30 días, aviso de privacidad y botón "borrar mi cuenta".
Utilidad social	Categorías del reto; integración con instituciones (rol validador y feed CAP).

3 La solución propuesta

3.1 Arquitectura general

ALERTA CERCA tiene cuatro piezas: la app, el backend en Supabase, el panel de validadores y los canales alternos. El diagrama muestra cómo viaja la información: los reportes entran al backend, el backend decide a quién avisar y la alerta sale por FCM hacia los teléfonos cercanos y por Telegram hacia los suscriptores.



3.2 Las cuatro piezas

Pieza	Qué hace	Quién la construye
App móvil Flutter (Android e iOS)	Recibir alertas; mapa de alertas cercanas; reportar en 3 pasos; confirmar ("yo también lo vi"); guardar "mis zonas" (casa, escuela, trabajo); compartir con contexto.	R3 pantallas R4 sistema
Backend Supabase	Base de datos PostgreSQL + PostGIS; cuentas (Auth); fotos (Storage); funciones en TypeScript (Edge Functions); tareas periódicas (pg_cron); actualizaciones en vivo (Realtime).	R1 datos R2 funciones
Panel de validadores Flutter Web	CCE, Protección Civil y escuelas revisan reportes, verifican, ajustan el radio, descartan, cierran y emiten alertas oficiales.	R5
Canales alternos	Bot de Telegram (el usuario comparte su ubicación y recibe alertas de su zona) y feed CAP para autoridades.	R2

3.3 Tecnologías y por qué

Componente	Tecnología	Por qué la elegimos	Alternativa válida
App móvil	Flutter (Dart)	Un código para Android, iOS y web; muchos paquetes de mapas y push.	React Native + Expo
Mapas	flutter_map + OpenStreetMap	Gratis, sin llave ni tarjeta.	Google Maps (requiere llave)
Base de datos	PostgreSQL + PostGIS	"¿Quién está a menos de X metros?" es una consulta SQL con índice espacial. El equipo ya sabe SQL.	Firestore + geohash (consultas más limitadas)
Cuentas	Supabase Auth	Usuarios anónimos para recibir; teléfono para reportar.	Firebase Auth
Fotos	Supabase Storage	Carpeta privada con enlaces temporales.	Cloudinary
Lógica en la nube	Edge Functions (TypeScript)	Incluidas en el plan gratuito, junto a la base de datos.	Cloud Functions (requiere plan de pago)
Tareas periódicas	pg_cron + pg_net	El radio dinámico y la expiración corren dentro de la base de datos.	Cron externo
Push	Firebase Cloud Messaging (API HTTP v1)	Gratis; Android, iOS y web; no depende del operador.	OneSignal
Panel	Flutter Web	Mismo lenguaje y modelos que la app.	React o HTML + supabase-js
Canal alternativo	Bot de Telegram	API gratuita; permite compartir ubicación.	WhatsApp Business (de paga)
Integración	CAP 1.2 (XML)	Estándar internacional de alertas públicas.	—

Por qué

Supabase nos da PostgreSQL de verdad: el corazón del sistema ("¿a quién le aviso?") es una consulta espacial que se escribe, se prueba y se optimiza en SQL. Firebase solo lo usamos para lo que hace mejor que nadie: entregar notificaciones push gratis.

4 Diseño detallado: lo que nos diferencia

4.1 Categorías y niveles

Cada categoría tiene un **nivel** (cómo suena), si **requiere validación** antes de difundirse, sus **escalones de radio** y su **vigencia**. Todo vive en tablas, no en el código: el CCE o Protección Civil pueden ajustar valores sin programar.

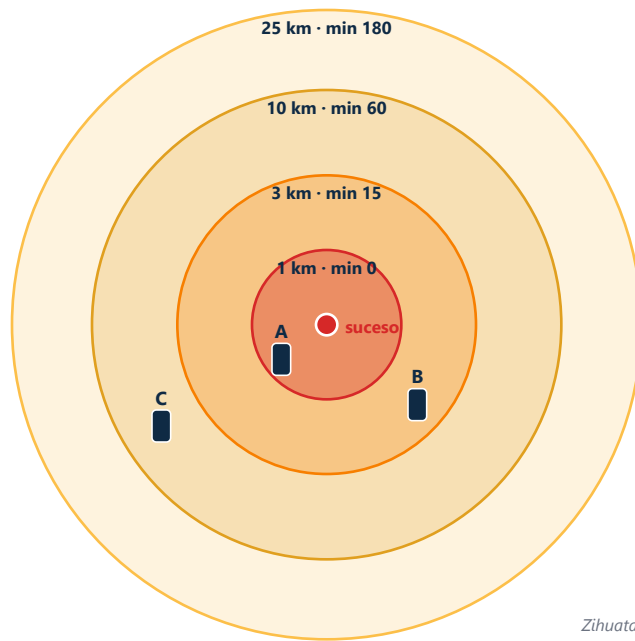
Categoría	Nivel	Validación previa	Escalones de radio (minuto → radio)	Vigencia
Menor desaparecido o posible sustracción	4	Sí	0 → 1 km · 15 → 3 km · 60 → 10 km · 180 → 25 km	72 h
Persona desaparecida	3	Sí	0 → 1 km · 30 → 3 km · 120 → 10 km	72 h
Adulto mayor o persona vulnerable extraviada	3	Sí	0 → 1 km · 30 → 3 km · 120 → 10 km	48 h
Robo de vehículo	3	No	0 → 3 km · 20 → 10 km · 60 → 25 km	24 h
Asalto o situación de riesgo	3	No	0 → 1 km · 30 → 2 km	2 h
Incendio	3	No	0 → 1 km · 20 → 3 km	6 h
Inundación	3	No	0 → 2 km · 60 → 5 km	12 h
Accidente o emergencia	2	No	0 → 1 km	3 h
Riesgo ambiental	2	No	0 → 3 km · 60 → 10 km	24 h
Evacuación	4	Solo instituciones	0 → 2 km (o el que fije la institución)	12 h
Fenómeno natural	4	Solo instituciones	0 → 10 km	24 h
Otra situación de riesgo	1	No	0 → 1 km	6 h

Nivel	Cómo se comporta en el teléfono
4 · Crítica	Canal de máxima importancia: sonido de alarma, vibración larga, aparece sobre otras apps.
3 · Alta	Notificación emergente con sonido.
2 · Media	Notificación normal.
1 · Informativa	Silenciosa; aparece en la lista y en el mapa.

Valores iniciales propuestos para la demo; deben revisarse con el CCE y Protección Civil.

4.2 Radio dinámico en anillos

Una alerta empieza con un radio pequeño y crece según los escalones de su categoría. Cada 15 segundos, una tarea programada (pg_cron) revisa las alertas activas: si el radio permitido creció, llama a la función notificar, que **solo envía a los teléfonos que todavía no la han recibido** (tabla entregas). Así nadie recibe la misma alerta dos veces.

**Ejemplo: menor desaparecido (verificado)****min 0 → radio 1 km**

A (300 m) recibe la alerta al instante

min 15 → radio 3 km

B (2.6 km) la recibe; A NO la recibe de nuevo

min 60 → radio 10 km

C (6 km) la recibe

min 180 → radio 25 km

cobertura máxima de la categoría

nunca → radio —

D (76 km) no la recibe: está lejos y no le sirve



Cada anillo se envía una sola vez (tabla entregas).

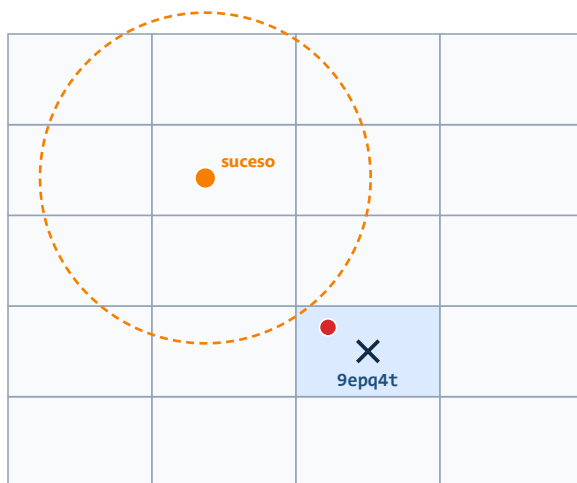
En la demo, factor_tiempo = 30: 1 min real = 30 min.

Zihuatanejo, 76 km

- **El reloj arranca cuando la alerta se publica** (al crearse si es un reporte no confirmado, o al verificarse si estaba en revisión).
- **Tope por confianza:** una alerta no confirmada llega como máximo a 1 km; corroborada por 3 vecinos, a 3 km; verificada por una institución, usa todos los escalones.
- **Radio manual:** el validador puede fijar un radio (por ejemplo, una evacuación de 2 km exactos).
- **Modo demo:** la tabla config tiene un factor_tiempo. Con 30, un minuto real equivale a 30 minutos: el escalón de 15 minutos llega en 30 segundos.

4.3 Privacidad por diseño: celdas geohash

Un **geohash** divide el mapa en cuadros con nombre. Con 6 caracteres, cada cuadro mide aproximadamente 1.2×0.6 km. El centro de Lázaro Cárdenas cae en la celda 9epq4t. El teléfono calcula su propia celda y **solo envía esos 6 caracteres**, y únicamente cuando cambia de cuadro. El servidor nunca recibe coordenadas de las personas.

Cada cuadro mide $\sim 1.2 \times 0.6$ km (geohash de 6 caracteres)● **Ubicación real: solo vive en el teléfono**✗ **Centro de la celda: lo único que sabe el servidor**

- 1 El teléfono calcula su celda (p. ej. 9epq4t) y la envía solo cuando cambia de cuadro.
- 2 El servidor busca celdas cuyo centro esté a menos de radio + 700 m del suceso (margen de celda).
- 3 El teléfono calcula la distancia exacta y decide: dentro del radio = alarma; en el margen = aviso normal.

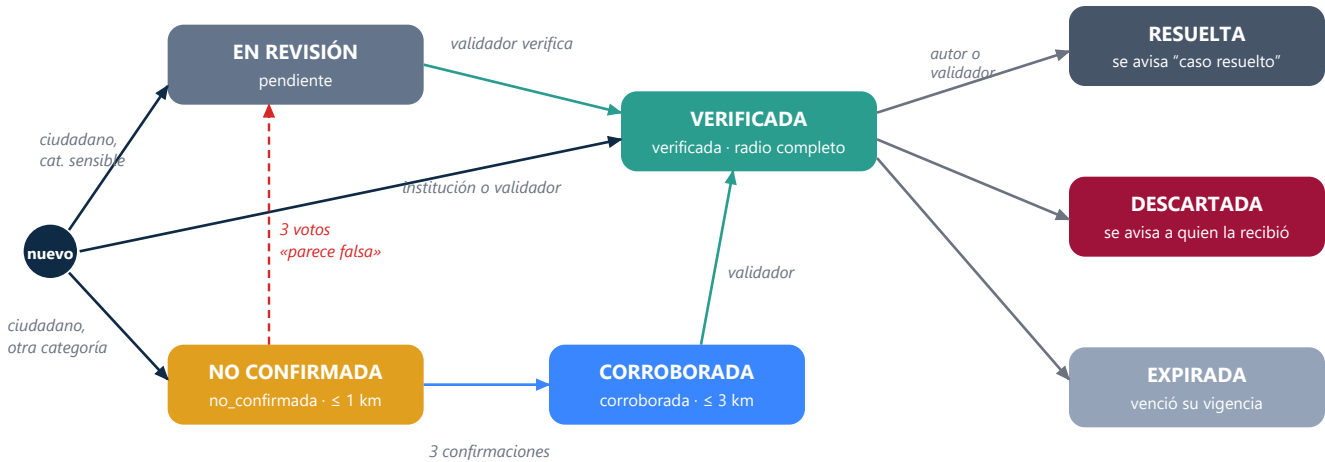
Resultado: nadie puede reconstruir tus recorridos aunque lea la base de datos completa.

Dato	¿Se guarda en el servidor?	Detalle
Ubicación exacta del teléfono	No	Solo en el teléfono (para calcular la distancia exacta).
Celda de ~1 km del teléfono	Sí	Se sobrescribe al moverse: no hay historial.
Historial de recorridos	No	No existe en ninguna tabla.
Zonas favoritas (casa, escuela)	Solo su celda	Nombre de la zona + celda de ~1 km.
Ubicación del suceso reportado	Sí, exacta	Es necesaria para la alerta; es un lugar, no una persona.
A quién se le envió cada alerta	Sí, 30 días	Para no repetir envíos y medir; se borra sola.
Teléfono de quien reporta	Sí (Auth)	Nunca se muestra a otros usuarios.
Metadatos de fotos (GPS, modelo)	No	Se eliminan en el teléfono antes de subir la foto.

Por qué

Usamos un margen de 700 m (la mitad de la diagonal de una celda) para **no dejar fuera** a nadie que esté dentro del radio. A cambio, algunos teléfonos un poco más lejos también la reciben; su app calcula la distancia exacta y les muestra un aviso normal en vez de una alarma.

4.4 Estados de una alerta (confiabilidad)



Descartar, resolver y expirar aplican a cualquier estado activo. Solo VERIFICADA usa todos los escalones de radio.

Estado	Cómo se ve en la app	Quién la recibe
En revisión (pendiente)	Solo la ve su autor: "Tu reporte está en revisión".	Solo validadores (push inmediato).
No confirmada	Insignia ámbar: "Reporte ciudadano sin confirmar".	Vecinos a ≤ 1 km.
Corroborada	Insignia azul: "Confirmada por 3 o más vecinos".	Hasta 3 km.
Verificada	Insignia verde: "Validada por Protección Civil" (u otra institución).	Todos los escalones.
Resuelta / Descartada	Gris, con el motivo; la foto deja de mostrarse.	Se avisa a quienes la recibieron.

4.5 Contra alertas falsas, maliciosas, duplicadas o viejas

Problema	Mecanismo
Cuentas falsas en masa	Para reportar se necesita una cuenta verificada con número de teléfono (1 número = 1 cuenta). Para recibir no se pide nada.
Spam	Máximo 3 reportes por hora por cuenta.
Usuarios con mal historial	Reputación: +1 si su reporte se verifica, -2 si se descarta. Con reputación menor a -2, sus reportes van directo a revisión.
Duplicados	Misma categoría, a menos de 500 m y en menos de 30 min: se suma como confirmación en vez de crear otra alerta.
Información falsa	Tres votos de "parece falsa" la regresan a revisión.
Información vieja	Vigencia por categoría y expiración automática.
Alertas resueltas que siguen circulando	Aviso de cierre a todos los que la recibieron; en la app aparece "RESUELTA".
Categorías graves	Personas y menores requieren validación previa; evacuaciones y fenómenos naturales solo los emiten instituciones.
Rendición de cuentas	Bitácora de quién verificó, descartó, ajustó o cerró cada alerta.
Reenvíos sin contexto	El texto de "Compartir" incluye fecha, hora y estado ("no confirmada"), para que no circule como si fuera oficial.

4.6 El riesgo que pocos equipos van a ver

Una alerta falsa de "persona desaparecida" puede usarse para **encontrar a alguien que no quiere ser encontrado**: por ejemplo, una persona que huyó de violencia familiar. Si la app difundiera su foto a toda la ciudad, estaríamos ayudando al agresor. Por eso:

- Las categorías de personas y menores **nunca se difunden sin validación**.
- El validador contacta a quien reporta y, de preferencia, pide el folio del reporte al 911 o de la denuncia.
- Nunca se publican domicilios; la foto se muestra solo con consentimiento del tutor o familiar.
- Quien reporta queda identificado (cuenta verificada) y todo queda en la bitácora.

Para el pitch

Mencionarlo en la presentación muestra que pensamos en el daño que la herramienta podría causar, no solo en el beneficio. Es el tipo de detalle que distingue a un equipo maduro.

4.7 Cómo se ven las notificaciones



Tipo	Ejemplo de título	Ejemplo de texto
Nueva (nivel 4)	MENOR DESAPARECIDO	A 450 m de ti · Verificada · Niño de 8 años, playera roja, visto frente al mercado hace 10 min.
Nueva (nivel 3)	INCENDIO	A 1.2 km de ti · No confirmada · Humo en bodega de la colonia Centro.
Actualización	AHORA VERIFICADA	Incendio en bodega: Protección Civil confirmó el reporte.
Cierre	RESUELTA	El menor fue localizado. Gracias por tu ayuda.
Descarte	DESCARTADA	La información no pudo confirmarse.

5 ¿Se puede alertar a un celular sin app?

La sección 13 del reto pide investigar las limitaciones para alertar a cualquier celular sin que exista una app instalada, sin autorización del usuario o sin integración con los operadores. Esta es nuestra respuesta:

Mecanismo	¿Llega sin instalar nada?	¿Gratis?	¿Independiente del operador?	¿Quién puede operarlo?	Veredicto
Cell Broadcast	Sí, a todos los celulares encendidos en la zona	Sí, para el usuario	No: lo transmiten las antenas de cada operador	Solo autoridades con los operadores	Ideal, pero fuera de nuestro alcance → integración futura vía CAP
SMS por ubicación	Sí	No: se paga cada mensaje	No	Operadores y agregadores	No cumple gratuidad
Push con app (FCM)	No: requiere la app	Sí	Sí: viaja por internet	Cualquiera	Núcleo de nuestra solución
App web (PWA) con notificaciones	No requiere tienda, pero sí entrar al sitio y aceptar	Sí	Sí	Cualquiera	Complemento; sin ubicación en segundo plano; en iPhone hay que agregarla a inicio
Bot de Telegram	Requiere Telegram (ya instalado en muchos teléfonos)	Sí	Sí	Cualquiera	Implementado como canal alternativo
WhatsApp Business	Requiere WhatsApp (casi universal)	No: Meta cobra por mensaje	Sí	Cualquiera con cuenta verificada	Futuro, si hay patrocinio
Redes sociales	Requiere seguir cuentas	Sí	Sí	Cualquiera	Ya existe y no es geográfico: es justo el problema

Dato para el pitch: México empezó a usar Cell Broadcast para mandar la alerta sísmica a los celulares en 2025. Confirмен el estado actual con fuentes oficiales antes de presentarlo.

Conclusión

Ningún particular puede enviar alertas a todos los celulares sin una app: el único canal universal (Cell Broadcast) lo controlan las autoridades y los operadores. Por eso proponemos: **(1)** una app gratuita como canal principal, **(2)** Telegram para quien no quiera instalar otra app y **(3)** una salida CAP para que, cuando una autoridad lo decida, las alertas verificadas de ALERTA CERCA puedan entrar a sus sistemas oficiales, incluido Cell Broadcast.

5.1 Estrategia de adopción: una app solo sirve si la tienen

Como dependemos de que la gente instale la app, la adopción es parte de la solución técnica:

- **Empresas del CCE:** que promuevan la app entre sus empleados (miles de personas que se mueven por la ciudad todos los días).
- **Escuelas:** los padres registran la escuela como "zona" y reciben alertas de ahí aunque estén en el trabajo.
- **Comercios:** código QR en mostradores ("Recibe alertas de tu colonia") que lleva a la tienda o al bot de Telegram.
- **Medios locales:** difundir el bot de Telegram como alternativa sin instalación.

Etapas	Canales	Alcance
Hoy (prototipo)	App + bot de Telegram + feed CAP	Entorno controlado del hackatón

Etapa	Canales	Alcance
Piloto	App en Google Play, empresas del CCE, escuelas, QR en comercios	Colonias seleccionadas de Lázaro Cárdenas
Futuro	Integración CAP con Protección Civil; Cell Broadcast vía autoridades; WhatsApp patrocinado	Toda la ciudad y otros municipios

6 Cómo cumplimos cada punto del reto

Esta tabla es para los jueces y para nosotros: cada objetivo específico del reto tiene una respuesta concreta y un momento de la demo donde se ve.

#	Objetivo específico	Cómo lo cumple ALERTA CERCA	Dónde se demuestra
1	Compatible con los principales sistemas operativos	Flutter: Android e iOS con el mismo código; web para el panel; Telegram en cualquier teléfono.	Demo en Android; código listo para iOS
2	Alertas vinculadas a una ubicación	Punto geográfico por GPS o pin en el mapa.	Pantalla Reportar
3	Niveles y categorías	12 categorías y 4 niveles en la tabla categorías.	Colores del mapa
4	Radios de 1, 3, 5, 10 o más km	Escalones por categoría y radio manual del validador.	Panel: Ajustar radio
5	Radio dinámico	pg_cron + radio_permitido() + tabla entregas (anillos).	B y C la reciben después
6	Foto, descripción, ubicación, fecha, hora, características	Campos de la alerta, Storage privado y casilla de consentimiento.	Detalle de alerta
7	Identificar el nivel de confiabilidad	Cuatro estados de confianza con insignias de color.	Insignias
8	Diferenciar verificadas de no confirmadas	Insignia, radio distinto y texto en la notificación.	Reporte ciudadano vs. verificado
9	Prevenir falsas, maliciosas, duplicadas o viejas	Diez mecanismos (sección 4.5).	Reporte duplicado en vivo
10	Cancelar o cerrar una alerta	Resolver o descartar, con aviso a quienes la recibieron.	Cierre al final
11	Privacidad y protección de datos	Celdas de ~1 km, RLS, fotos sin metadatos, borrado automático, aviso de privacidad.	Tabla dispositivos en vivo
12	Escalar e integrarse con autoridades	Roles de institución y validador, feed CAP, diseño multi-ciudad.	Feed CAP en el navegador
13	Llegar a más dispositivos sin depender de un operador	Push por internet, Telegram y el análisis de la sección 5.	Bot de Telegram con el jurado

6.1 Restricciones (sección 13 del reto)

Restricción	Cómo se cumple
Gratuidad para el usuario	El usuario no paga nada; el prototipo corre en planes gratuitos.
Independencia del operador	Todo viaja por internet (datos o wifi); no usamos SMS para alertar.
Compatibilidad	Android, iOS (mismo código), web y Telegram.
Consentimiento	Cada permiso se explica antes de pedirlo; la ubicación en segundo plano es opcional.
Uso responsable de la geolocalización	Celdas de ~1 km, sin historial.
Protección de datos personales	Sección 12 de este documento.
Seguridad de la información	RLS, funciones del servidor, HTTPS, llaves en secretos (Vault).
Validación de alertas	Estados, validadores y bitácora.
Prevención de información falsa	Sección 4.5.
Cierre o cancelación	Estados resuelta, descartada y expirada.

Restricción	Cómo se cumple
Escalabilidad	Índices espaciales y prueba de carga con 5,000 dispositivos (Fase 5).

7 Organización del equipo

7.1 Roles y responsabilidades

Cada integrante tiene un rol principal y un compañero de apoyo. Los pares (R1–R2 y R3–R4) se revisan el código entre sí; R5 coordina y prueba todo.

Rol	Responsable de	Perfil ideal	Apoya a
R1 Base de datos y backend geográfico (líder técnico)	Proyecto Supabase; esquema; PostGIS; seguridad RLS; funciones SQL (crear_reporte, radio_permitido, dispositivos_objetivo, validar_alerta); pg_cron; prueba de carga.	SQL fuerte (quien domine bases de datos).	R2
R2 Servicios en la nube y notificaciones	Proyecto Firebase; Edge Functions en TypeScript: notificar, telegram-webhook y cap; secretos; bot de Telegram.	JavaScript o TypeScript, APIs.	R1
R3 App móvil: pantallas	Bienvenida y permisos, mapa de inicio, detalle de alerta, reportar en 3 pasos, insignias, compartir, diseño visual.	Flutter, gusto por el diseño.	R4
R4 App móvil: sistema	Firebase en la app, geohash, ubicación, registro del dispositivo, recepción de push y canales, mis zonas, modo demo, cuenta con teléfono.	Flutter, Android.	R3
R5 Panel, producto y presentación	Panel de validadores (Flutter Web); aviso de privacidad; plan de pruebas; tablero de tareas; guion de la demo; diapositivas; video de respaldo.	Organización, comunicación, Flutter básico.	Todos

Liderazgo

R1 es el líder técnico (decide arquitectura y nombres) y R5 coordina (agenda, tablero, tiempos). Separar esos dos papeles evita que una sola persona se sature.

7.2 Contratos entre roles: acuérdenlos antes de programar

Para que cinco personas trabajen en paralelo sin bloquearse, primero se fijan los "contratos": las funciones que expone la base de datos y el formato del mensaje push. Con esto, R3, R4 y R5 pueden programar sus pantallas con datos de prueba aunque el backend no esté terminado.

Función (RPC)	Parámetros	Devuelve	La usa
registrar_dispositivo	p_token, p_plataforma, p_celda	—	R4
crear_reporte	p_categoria, p_titulo, p_descripcion, p_referencia, p_lat, p_lon, p_foto_path	{alerta_id, estado} o {duplicada_de}	R3, R5
alertas_cercanas	p_celda, p_radio_m (25000)	lista de alertas	R3
confirmar_alerta	p_alerta, p_tipo ('confirmo' 'ya_no_esta' 'parece_falsa')	—	R3
validar_alerta	p_alerta, p_accion ('verificar' 'descartar' 'resolver' 'ajustar_radio'), p_motivo, p_radio_m	—	R5 (y R3 para "resolver" del autor)

Función (RPC)	Parámetros	Devuelve	La usa
borrar_mi_cuenta	—	—	R3

Campo del push (data)	Ejemplo	Descripción
tipo	nueva	nueva · actualizacion · cierre · validacion
alerta_id	b3c1...	Identificador para abrir el detalle
categoria / nivel	incendio / 3	Para el icono y el canal de notificación
estado	no_confirmada	Para la insignia
titulo / cuerpo	INCENDIO / Humo en bodega...	Textos ya listos para mostrar
lat / lon	17.9581 / -102.1942	Ubicación del suceso (para calcular la distancia en el teléfono)
radio_m	1000	Radio con el que se envió
foto	1	"1" si tiene foto

FCM exige que todos los valores de data sean texto: los números viajan como cadenas ("1000"), y la app los convierte.

7.3 Estructura del repositorio

alerta-cerca/	
├─ app/	# App móvil Flutter (R3, R4)
│ └─ lib/	
│ │ └─ main.dart	
│ │ └─ config.dart	# URL y anon key de Supabase (públicas)
│ │ └─ nucleo/	# geohash, ubicación, notificaciones, sesión
│ │ └─ datos/	# modelos y llamadas a Supabase
│ │ └─ pantallas/	# bienvenida, inicio, detalle, reportar, zonas, ajustes
├─ panel/	# Panel de validadores, Flutter Web (R5)
├─ supabase/	# Backend (R1, R2)
│ └─ migrations/	# 001_esquema.sql, 002_triggers.sql, 003_seguridad.sql ...
│ └─ functions/	
│ │ └─ _compartido/geohash.ts	
│ │ └─ notificar/index.ts	
│ │ └─ telegram-webhook/index.ts	
│ │ └─ cap/index.ts	
└─ seed.sql	# categorías, escalones y datos de la demo
└─ docs/	# esta propuesta, aviso de privacidad, guion, capturas

7.4 Reglas de trabajo

- **Rama main protegida.** Cada quien trabaja en ramas feat/r3-pantalla-detalle y abre un Pull Request que revisa su compañero de apoyo.
- **Commits pequeños** y con mensaje claro en español: "Agrega pantalla de detalle con insignia de estado".
- **Nunca subir secretos:** la service_role key, la cuenta de servicio de Firebase y el token del bot van en secretos de Supabase o en archivos ignorados por git. La anon key sí puede ir en la app.
- **Tablero:** GitHub Projects con columnas Por hacer / En curso / En revisión / Hecho. Cada paso de la sección 8 es una tarjeta.
- **Juntas cortas:** 10 minutos cada 4 horas durante el hackatón: qué terminé, qué sigue, en qué estoy atorado.
- **Regla de los 30 minutos:** si alguien lleva media hora atorado, pide ayuda. Siempre.

8 Plan de implementación paso a paso

El trabajo se divide en una fase de preparación (antes del hackatón) y cinco fases durante el evento. Las horas suponen un hackatón de unas 40 horas; si dura menos, usen la tabla de prioridades de la sección 8.2. Cada paso indica **quién lo hace** (insignia de color), **qué hacer exactamente** y **cómo saber que está terminado**.

8.1 Cronograma por rol

Rol	Fase 0 Preparación (antes)	Fase 1 Cimientos (h 0–6)	Fase 2 Flujo principal (h 6–16)	Fase 3 Inteligencia (h 16–26)	Fase 4 Canales (h 26–34)	Fase 5 Pruebas y demo (h 34–40)
R1	Supabase, extensiones	Esquema, triggers, RLS, catálogo	Funciones de reporte y geográficas; trigger a notificar	pg_cron, confirmaciones, validar_alerta, expiración	Métricas, retención, Advisors	Prueba de carga, datos de la demo
R2	Firebase, BotFather, función "hola"	Secretos, cuenta de servicio, esqueleto de notificar	notificar completa (FCM)	Probar cierre, actualización y validadores	Bot de Telegram y feed CAP	Soporte y bitácoras
R3	Flutter y emulador listos	Proyecto, navegación y tema	Inicio con mapa, Detalle, Reportar	Insignias, confirmar, compartir, bienvenida y permisos	Pulido visual	Capturas y ensayo
R4	Flutter y emulador con Google Play	Firebase en la app, sesión anónima	Geohash, celda, recepción de push, canales	Mis zonas, modo demo, cuenta con teléfono	Segundo plano (opcional)	Teléfonos de la demo
R5	Tablero y documentos	Proyecto del panel, inicio de sesión	Cola de validación y mapa	Acciones, tiempo real, alerta oficial	Aviso de privacidad, diapositivas	Pruebas, guion y video

8.2 Prioridades: qué hacer primero si falta tiempo

Prioridad	Incluye
Indispensable sin esto no hay demo	Esquema y RLS; registro del dispositivo con celda; crear_reporte; notificar; recepción del push con distancia; mapa, detalle y reportar; panel con verificar y resolver; radio dinámico; aviso de cierre.
Importante	Estados no confirmada y corroborada; duplicados; límite de reportes; fotos sin metadatos; modo demo; pantalla de bienvenida con permisos; cuenta con teléfono.
Deseable	Bot de Telegram; feed CAP; mis zonas; métricas en el panel; actualización en segundo plano; prueba de carga.
Fuera del prototipo	Publicar en las tiendas; iOS en teléfono real; Cell Broadcast, SMS o WhatsApp; corredores viales; varios idiomas.

⚠ Ojo

Congelen funciones nuevas **4 horas antes del cierre**. Desde ese momento solo se corrigen errores, se prueba la demo y se ensaya el pitch. Una demo que funciona vale más que tres funciones a medias.

8.3 Fase 0 · Preparación (antes del hackatón)

Objetivo: Llegar al evento con todas las cuentas creadas, las herramientas instaladas y un "hola mundo" funcionando en cada computadora. Calculen de 3 a 4 horas por persona durante la semana previa.

Paso 0.1 **Todos** Crear las cuentas

Cuenta	Quién	Dónde	Notas
GitHub	Todos	github.com	R1 crea el repositorio y agrega a los demás.
Supabase	R1 (invita a R2 y R5)	supabase.com	Entrar con la cuenta de GitHub. Plan Free.
Firebase	R2 (invita a R4)	console.firebase.google.com	Cuenta de Google. Plan Spark (gratis).
Bot de Telegram	R2	@BotFather en Telegram	Ver Paso 0.6.
Apple Developer	Nadie	—	No hace falta para el prototipo (cuesta 99 USD al año).

Paso 0.2 **Todos** Instalar las herramientas (Windows)

1. **Git** (git-scm.com) y **Visual Studio Code** con las extensiones *Flutter* y *Dart*. R1 y R2 agreguen la extensión *Deno* (para TypeScript de las Edge Functions).
2. **Android Studio**: instalen el Android SDK y creen un emulador en *Device Manager* → *Create device* → *Pixel* con una imagen de sistema que diga **"Google Play"** (sin Google Play, FCM no funciona en el emulador).
3. **Flutter SDK**: sigan la guía oficial de docs.flutter.dev para Windows y Android. Al final, `flutter doctor` debe salir en verde en Flutter, Android toolchain y Android Studio.
4. **R1 y R2**: Supabase CLI. **R4**: Node.js LTS, Firebase CLI y FlutterFire CLI (comandos abajo).

```
flutter doctor                                # revisar que todo esté en verde
flutter doctor --android-licenses             # aceptar licencias de Android

# R1 y R2 - Supabase CLI con Scoop (en PowerShell)
irm get.scoop.sh | iex                       # instala Scoop si no lo tienen
scoop bucket add supabase https://github.com/supabase/scoop-bucket.git
scoop install supabase
supabase --version

# R4 - Firebase CLI y FlutterFire CLI (requieren Node.js LTS)
npm install -g firebase-tools
dart pub global activate flutterfire_cli
```

Paso 0.3 **R1** Crear el repositorio

1. En GitHub: *New repository* → alerta-cerca, privado, con README.
2. Settings → Collaborators: agregar a los 4 compañeros.
3. Settings → Branches → *Add rule* para main: pedir Pull Request y 1 aprobación.
4. Crear las carpetas de la sección 7.3 y un `.gitignore` con lo de abajo.

```
# .gitignore (raíz del repositorio)
.env
*.env
**/cuenta-servicio*.json      # Llave privada de Firebase: NUNCA al repositorio
supabase/.temp/
supabase/.branches/
**/build/
**/.dart_tool/
.idea/
.vscode/
```

Paso 0.4 R1 Crear el proyecto de Supabase

1. supabase.com → *New project*. Nombre: alerta-cerca. Contraseña de la base de datos: una fuerte, guardada en un gestor de contraseñas. Región: la más cercana disponible (por ejemplo, East US). Plan: Free.
2. *Project Settings* → *API*: copien la **Project URL**, la **anon key** (pública, va en la app) y la **service_role key** (SECRETA: solo para el servidor, nunca en la app ni en git).
3. *Authentication* → *Sign In / Providers*: activen **Allow anonymous sign-ins**, **Email** (para cuentas de validadores) y **Phone** (para quien reporta).
4. En Phone, agreguen **números de prueba** con su código fijo (por ejemplo, 525511111111 → 123456) para la demo. Si Supabase pide un proveedor de SMS para guardar, creen una cuenta de prueba de Twilio (trae crédito gratis).
5. *Database* → *Extensions*: activen postgres, pg_cron y pg_net (también se hace con SQL en el Paso 1.1).
6. *Organization* → *Team*: inviten a R2 y R5.

⚠ Ojo

Los proyectos gratuitos de Supabase **se pausan tras 7 días sin actividad**. Entren al panel y ejecuten una consulta el día anterior al hackatón y el día de la final.

Paso 0.5 R2 Crear el proyecto de Firebase (solo para notificaciones)

1. console.firebase.google.com → *Agregar proyecto* → alerta-cerca. Desactiven Google Analytics (no lo necesitamos).
2. La app Android se registra sola en el Paso 1.7 con flutterfire configure. Su paquete será mx.alertacerca.alerta_cerca.
3. *Configuración del proyecto* → *Cuentas de servicio* → *Generar nueva clave privada*. Se descarga un JSON. **Es secreto**: pásenlo a R2 por un medio privado; irá en los secretos de Supabase.
4. *Configuración* → *Cloud Messaging*: verifiquen que la **Firebase Cloud Messaging API (V1)** esté habilitada.

Paso 0.6 R2 Crear el bot de Telegram

1. En Telegram, abran **@BotFather** y envíen /newbot.
2. Nombre: *ALERTA CERCA Demo*. Usuario: debe terminar en "bot", por ejemplo alerta_cerca_demo_bot (si está ocupado, agreguen el nombre del equipo).
3. Guarden el **token** que les da (secreto).
4. /setdescription: "Recibe alertas de emergencia cerca de tu zona. Prototipo HackaITLAC 2026. No sustituye al 911."
5. Generen un código QR del enlace https://t.me/<usuario_del_bot> para la demo.

Paso 0.7 R5 Tablero, documentos y agenda

- GitHub → Projects → *New project (Board)* con columnas Por hacer / En curso / En revisión / Hecho. Una tarjeta por cada paso de esta sección, asignada a su rol.
- Carpeta compartida con: esta propuesta, el PDF del reto, capturas y la plantilla de diapositivas.
- Calendario del evento con las juntas cortas cada 4 horas y la hora de congelar funciones.

Paso 0.8 **Todos** "Hola mundo" de cada quien

Rol	Prueba	Resultado esperado
R3 R4	flutter create hola, luego flutter run en el emulador	Aparece la app del contador.
R1	SQL Editor: select postgis_full_version();	Devuelve la versión de PostGIS.
R2	supabase functions new hola y desplegarla (Paso 1.9)	La URL responde "Hello".
R5	flutter run -d chrome	La app abre en Chrome.

Lista de verificación de la Fase 0

- ☐ Las 5 personas tienen acceso al repositorio, y R1, R2 y R5 al proyecto de Supabase.
- ☐ Emuladores con Google Play funcionando en las computadoras de R3, R4 y R5.
- ☐ URL, anon key y service_role key de Supabase guardadas (la última, en privado).
- ☐ JSON de la cuenta de servicio de Firebase en manos de R2.
- ☐ Token del bot de Telegram en manos de R2 y QR generado.
- ☐ Tablero con todas las tarjetas y responsables.

8.4 Fase 1 · Cimientos (horas 0 a 6)

Objetivo: base de datos completa y segura, app y panel que abren y se conectan a Supabase.

Paso 1.0 **Todos** Junta de arranque (30 minutos)

- Repasar la arquitectura (sección 3) y los **contratos** (sección 7.2). Nadie programa antes de esto.
- Confirmar los nombres: tablas, funciones, campos del push. Si alguien quiere cambiar un nombre, se decide ahora y R1 lo anota.
- Cada quien crea su rama y toma sus primeras tarjetas.

Paso 1.1 **R1** Extensiones y tipos

Abran *SQL Editor* en Supabase, peguen cada bloque y ejecútenlo. Guarden el mismo SQL en `supabase/migrations/001_esquema.sql` para que quede en el repositorio.

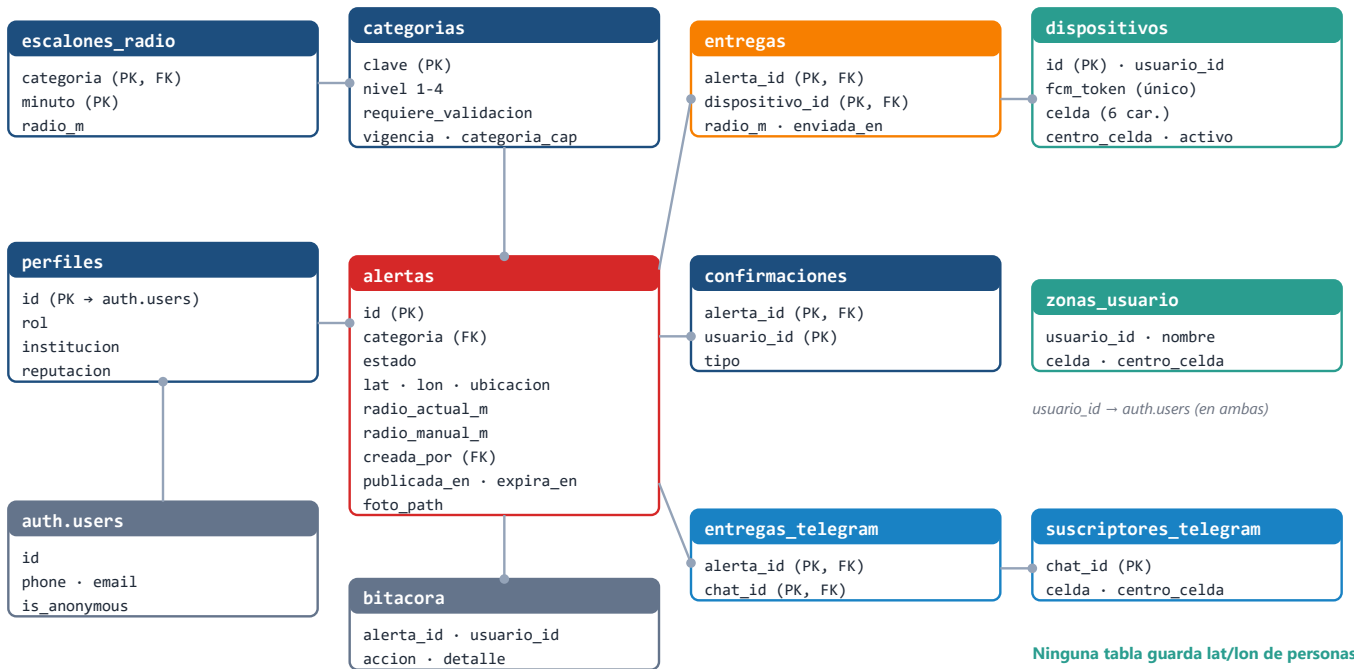
```
-- 001_esquema.sql (parte 1)
create extension if not exists postgis with schema extensions;
create extension if not exists pg_cron;
create extension if not exists pg_net;

create type categoria_alerta as enum (
  'menor_desaparecido', 'persona_desaparecida', 'persona_vulnerable', 'robo_vehiculo',
  'asalto', 'incendio', 'inundacion', 'accidente', 'riesgo_ambiental', 'evacuacion',
  'fenomeno_natural', 'otro');

create type estado_alerta as enum (
  'pendiente', 'no_confirmada', 'corroborada', 'verificada', 'resuelta', 'descartada', 'expirada');

create type rol_usuario as enum ('ciudadano', 'validador', 'institucion', 'admin');
```

Paso 1.2 R1 Tablas



```
-- 001_esquema.sql (parte 2)
create table perfiles (
  id          uuid primary key references auth.users on delete cascade,
  nombre      text,
  rol         rol_usuario not null default 'ciudadano',
  institucion text,
  reputacion  int not null default 0,
  creado_en   timestamptz not null default now()
);

create table categorias (
  clave          categoria_alerta primary key,
  nombre         text not null,
  nivel          smallint not null check (nivel between 1 and 4),
  requiere_validacion boolean not null default false,
  solo_institucion boolean not null default false,
  radio_max_no_conf_m int not null default 1000,
  vigencia       interval not null,
  categoria_cap  text not null
);

create table escalones_radio (
  categoria categoria_alerta references categorias,
  minuto    int not null,
  radio_m   int not null,
  primary key (categoria, minuto)
);

create table alertas (
  id          uuid primary key default gen_random_uuid(),
  categoria   categoria_alerta not null references categorias,
  titulo      text not null check (char_length(titulo) between 5 and 80),
  descripcion text check (char_length(descripcion) <= 1000),
  referencia  text,
  foto_path   text,
  lat         double precision not null check (lat between -90 and 90),
  lon         double precision not null check (lon between -180 and 180),
  ubicacion   geography(Point, 4326),
  estado      estado_alerta not null default 'pendiente',
  -- p. ej. 'Protección Civil Municipal'
  -- personas y menores
  -- evacuación, fenómenos naturales
  -- tope si no está confirmada
  -- Rescue, Fire, Security... (Paso 4.2)
  -- "frente al mercado municipal"
  -- ruta en Storage (bucket privado)
  -- La llena un trigger (Paso 1.3)

```

```

radio_actual_m int not null default 0,           -- hasta dónde ya se envió
radio_manual_m int,                             -- si el validador fija uno
creada_por      uuid references perfiles on delete set null,
verificada_por  uuid references perfiles on delete set null,
creada_en       timestampz not null default now(),
publicada_en    timestampz,                     -- arranca el reloj del radio dinámico
verificada_en   timestampz,
cerrada_en      timestampz,
expira_en       timestampz not null,
motivo_cierre   text
);
create index alertas_ubicacion_idx on alertas using gist (ubicacion);
create index alertas_estado_idx on alertas (estado);

create table dispositivos (
  id          uuid primary key default gen_random_uuid(),
  usuario_id  uuid not null references auth.users on delete cascade,
  fcm_token   text not null unique,
  plataforma  text not null check (plataforma in ('android', 'ios', 'web')),
  celda       text not null check (char_length(celda) = 6), -- geohash de ~1 km
  centro_celda geography(Point, 4326),             -- La llena un trigger
  activo      boolean not null default true,
  actualizado_en timestampz not null default now()
);
create index dispositivos_centro_idx on dispositivos using gist (centro_celda) where activo;

create table zonas_usuario (
  id          uuid primary key default gen_random_uuid(),
  usuario_id  uuid not null references auth.users on delete cascade,
  nombre      text not null,                      -- 'Casa', 'Escuela de Ana'
  celda       text not null check (char_length(celda) = 6),
  centro_celda geography(Point, 4326),
  unique (usuario_id, nombre)
);
create index zonas_centro_idx on zonas_usuario using gist (centro_celda);

create table entregas (
  alerta_id    uuid references alertas on delete cascade,
  dispositivo_id uuid references dispositivos on delete cascade,
  radio_m      int not null,
  enviada_en   timestampz not null default now(),
  primary key (alerta_id, dispositivo_id)
);

create table confirmaciones (
  alerta_id    uuid references alertas on delete cascade,
  usuario_id   uuid references auth.users on delete cascade,
  tipo        text not null check (tipo in ('confirmo', 'ya_no_esta', 'parece_falsa')),
  creada_en    timestampz not null default now(),
  primary key (alerta_id, usuario_id)
);

create table suscriptores_telegram (
  chat_id      bigint primary key,
  celda       text not null check (char_length(celda) = 6),
  centro_celda geography(Point, 4326),
  activo      boolean not null default true,
  creado_en    timestampz not null default now()
);

create table entregas_telegram (
  alerta_id    uuid references alertas on delete cascade,
  chat_id      bigint references suscriptores_telegram on delete cascade,
  enviada_en   timestampz not null default now(),
  primary key (alerta_id, chat_id)
);

create table bitacora (
  id          bigint generated always as identity primary key,
  alerta_id    uuid references alertas on delete cascade,

```

```

    usuario_id uuid,
    accion      text not null,
    detalle     jsonb,
    creada_en   timestampz not null default now()
);

create table config (
  id          int primary key default 1 check (id = 1), -- una sola fila
  factor_tiempo numeric not null default 1             -- demo: 30
);
insert into config default values;
```

⚠ Ojo

En Supabase, PostGIS vive en el esquema `extensions`, que ya está en el `search_path`. Si algún comando marca "type geography does not exist", escriban `extensions.geography`. Por la misma razón, todas las funciones de este documento llevan `set search_path = public, extensions`.

Paso 1.3 R1 Triggers: ubicaciones, celdas y perfiles automáticos

```

-- 002_triggers.sql
-- Convierte lat/lon de la alerta en un punto geográfico
create or replace function fijar_ubicacion_alerta() returns trigger
language plpgsql set search_path = public, extensions as $$
begin
    new.ubicacion := st_setsrid(st_makepoint(new.lon, new.lat), 4326)::geography;
    return new;
end $$;
create trigger alertas_ubicacion before insert or update of lat, lon on alertas
for each row execute function fijar_ubicacion_alerta();

-- Convierte la celda (geohash) en el punto central de ese cuadro
create or replace function fijar_centro_celda() returns trigger
language plpgsql set search_path = public, extensions as $$
begin
    new.centro_celda := st_setsrid(st_pointfromgeohash(new.celda), 4326)::geography;
    return new;
end $$;
create trigger dispositivos_celda before insert or update of celda on dispositivos
for each row execute function fijar_centro_celda();
create trigger zonas_celda before insert or update of celda on zonas_usuario
for each row execute function fijar_centro_celda();
create trigger telegram_celda before insert or update of celda on suscriptores_telegram
for each row execute function fijar_centro_celda();

-- Cada usuario nuevo (incluso anónimo) recibe su perfil de 'ciudadano'
create or replace function crear_perfil() returns trigger
language plpgsql security definer set search_path = public as $$
begin
    insert into public.perfiles (id) values (new.id);
    return new;
end $$;
create trigger al_crear_usuario after insert on auth.users
for each row execute function crear_perfil();
```

Comprueben que el geohash funciona igual en PostGIS que en la app (Paso 2.6):

```

select st_geohash(st_setsrid(st_makepoint(-102.1942, 17.9581), 4326), 6); -- debe dar 9epq4t
select st_astext(st_pointfromgeohash('9epq4t'));                        -- POINT(-102.189... 17.959...)
```

Paso 1.4 R1 Catálogo de categorías y escalones (seed.sql)

```
insert into categorias (clave, nombre, nivel, requiere_validacion, solo_institucion,
                      radio_max_no_conf_m, vigencia, categoria_cap) values
('menor_desaparecido', 'Menor desaparecido o posible sustracción', 4, true, false, 0, '72 hours', 'Rescue'),
('persona_desaparecida', 'Persona desaparecida', 3, true, false, 0, '72 hours', 'Rescue'),
('persona_vulnerable', 'Adulto mayor o persona vulnerable extraviada', 3, true, false, 0, '48 hours', 'Rescue'),
('robo_vehiculo', 'Robo de vehículo', 3, false, false, 1000, '24 hours', 'Security'),
('asalto', 'Asalto o situación de riesgo', 3, false, false, 1000, '2 hours', 'Security'),
('incendio', 'Incendio', 3, false, false, 1000, '6 hours', 'Fire'),
('inundacion', 'Inundación', 3, false, false, 1000, '12 hours', 'Met'),
('accidente', 'Accidente o emergencia', 2, false, false, 1000, '3 hours', 'Transport'),
('riesgo_ambiental', 'Riesgo ambiental', 2, false, false, 1000, '24 hours', 'Env'),
('evacuacion', 'Evacuación', 4, false, true, 0, '12 hours', 'Safety'),
('fenomeno_natural', 'Fenómeno natural', 4, false, true, 0, '24 hours', 'Geo'),
('otro', 'Otra situación de riesgo', 1, false, false, 1000, '6 hours', 'Other');

insert into escalones_radio (categoria, minuto, radio_m) values
('menor_desaparecido', 0, 1000), ('menor_desaparecido', 15, 3000),
('menor_desaparecido', 60, 10000), ('menor_desaparecido', 180, 25000),
('persona_desaparecida', 0, 1000), ('persona_desaparecida', 30, 3000), ('persona_desaparecida', 120, 10000),
('persona_vulnerable', 0, 1000), ('persona_vulnerable', 30, 3000), ('persona_vulnerable', 120, 10000),
('robo_vehiculo', 0, 3000), ('robo_vehiculo', 20, 10000), ('robo_vehiculo', 60, 25000),
('asalto', 0, 1000), ('asalto', 30, 2000),
('incendio', 0, 1000), ('incendio', 20, 3000),
('inundacion', 0, 2000), ('inundacion', 60, 5000),
('accidente', 0, 1000),
('riesgo_ambiental', 0, 3000), ('riesgo_ambiental', 60, 10000),
('evacuacion', 0, 2000),
('fenomeno_natural', 0, 10000),
('otro', 0, 1000);
```

Paso 1.5 R1 Seguridad: RLS, fotos y tiempo real

Con RLS (Row Level Security), cada consulta que llega desde la app se filtra fila por fila según quién la hace. Regla de oro: **la app nunca escribe directamente en alertas**; solo llama funciones del servidor que validan todo.

```
-- 003_seguridad.sql
create or replace function es_validador() returns boolean
language sql stable security definer set search_path = public as $$
    select exists (select 1 from perfiles
                  where id = auth.uid() and rol in ('validador', 'institucion', 'admin'));
$$;

alter table perfiles           enable row level security;
alter table categorias         enable row level security;
alter table escalones_radio    enable row level security;
alter table alertas            enable row level security;
alter table dispositivos       enable row level security;
alter table zonas_usuario     enable row level security;
alter table entregas           enable row level security;
alter table entregas_telegram  enable row level security; -- sin políticas: solo el servidor
alter table confirmaciones     enable row level security;
alter table suscriptores_telegram enable row level security; -- sin políticas: solo el servidor
alter table bitacora           enable row level security;
alter table config             enable row level security; -- sin políticas: solo el servidor

create policy "catalogo publico"    on categorias    for select using (true);
create policy "escalones publicos"  on escalones_radio for select using (true);
create policy "mi perfil"           on perfiles      for select using (id = auth.uid() or es_validador());
create policy "alertas visibles"    on alertas       for select using (
    estado in ('no_confirmada', 'corroborada', 'verificada', 'resuelta')
    or creada_por = auth.uid() or es_validador());
-- alertas no tiene políticas de insert/update: solo se modifica con crear_reporte() y validar_alerta()
create policy "mis dispositivos"    on dispositivos  for all
using (usuario_id = auth.uid()) with check (usuario_id = auth.uid());
create policy "mis zonas"           on zonas_usuario for all
using (usuario_id = auth.uid()) with check (usuario_id = auth.uid());
```

```
create policy "ver confirmaciones" on confirmaciones for select using (true);
create policy "entregas validadores" on entregas for select using (es_validador());
create policy "bitacora validadores" on bitacora for select using (es_validador());

-- Fotos: bucket privado 'fotos'; cada quien sube solo a su carpeta (su id de usuario)
insert into storage.buckets (id, name, public) values ('fotos', 'fotos', false);
create policy "subir mis fotos" on storage.objects for insert to authenticated
  with check (bucket_id = 'fotos' and (storage.foldername(name))[1] = auth.uid()::text);
create policy "ver fotos" on storage.objects for select to authenticated
  using (bucket_id = 'fotos');

-- El panel se actualiza solo cuando cambia una alerta
alter publication supabase_realtime add table alertas;
```

✓ Terminado cuando

Las 12 tablas aparecen en *Table Editor* con el candado de RLS activo; `select * from categorias;` devuelve 12 filas; *Advisors* → *Security* no marca tablas sin RLS.

Paso 1.6 R3 R4 Crear la app móvil

```
cd alerta-cerca
flutter create --org mx.alertacerca --project-name alerta_cerca --platforms android,ios app
cd app
flutter pub add supabase_flutter firebase_core firebase_messaging flutter_local_notifications
flutter pub add geolocator shared_preferences flutter_map latlong2 image_picker
flutter pub add flutter_image_compress url_launcher share_plus intl
flutter pub add workmanager # opcional: actualización en segundo plano (Paso 4.4)
```

`flutter pub add` instala la versión más reciente de cada paquete. Las firmas exactas de algunas funciones cambian entre versiones: si algo de este documento no compila, revisen el README y el ejemplo del paquete en `pub.dev`. Después del primer `flutter pub get`, suban el `pubspec.lock` para que todo el equipo use las mismas versiones.

Permisos en `android/app/src/main/AndroidManifest.xml` (dentro de `<manifest>`, antes de `<application>`):

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION"/> <!-- opcional -->
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
<uses-permission android:name="android.permission.VIBRATE"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
```

- En `android/app/build.gradle` (o `.kts`): `minSdk = 23` y activen *core library desugaring* como indica el README de `flutter_local_notifications`.
- Creen las carpetas `lib/nucleo`, `lib/datos` y `lib/pantallas`, y el archivo de configuración:

```
// lib/config.dart: La anon key es pública; La service_role NUNCA va en la app
const supabaseUrl = 'https://TU-PROYECTO.supabase.co';
const supabaseAnonKey = 'eyJhbGciOi...';
```

Paso 1.7 R4 Conectar Firebase y crear la sesión anónima

```
firebase login
flutterfire configure --project=alerta-cerca --platforms=android,ios
# crea lib/firebase_options.dart y android/app/google-services.json

// lib/main.dart
final navegador = GlobalKey<NavigatorState>();

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  FirebaseMessaging.onBackgroundMessage(alRecibirEnSegundoPlano); // Paso 2.7
```

```

await Supabase.initialize(url: supabaseUrl, anonKey: supabaseAnonKey);

final auth = Supabase.instance.client.auth;
if (auth.currentSession == null) {
  await auth.signInAnonymously(); // recibir alertas no requiere cuenta
}
await Notificaciones.inicializar(
  alTocar: (id) => navegador.currentState?.pushNamed('/alerta', arguments: id));
FirebaseMessaging.instance.onTokenRefresh
  .listen((_) => Ubicacion.actualizarMiCelda(forzar: true)); // Paso 2.6
runApp(const AlertaCercaApp());
}

```

✓ Terminado cuando

La app abre en el emulador y en Supabase, *Authentication* → *Users*, aparece un usuario "Anonymous"; en *Table Editor* → *perfiles* existe su fila con rol ciudadano.

Paso 1.8 R5 Crear el panel y las cuentas de validadores

```

cd alerta-cerca
flutter create --org mx.alertacerca --project-name panel_validadores --platforms web panel
cd panel
flutter pub add supabase_flutter flutter_map latlong2 intl
flutter run -d chrome

```

1. En Supabase, *Authentication* → *Users* → *Add user*: `validador1@example.com` con contraseña y *Auto Confirm User* activado. Repitan para `validador2`.
2. Denles el rol con SQL:

```

update perfiles set rol = 'validador', nombre = 'Validador 1',
  institucion = 'Protección Civil (demo)'
where id = (select id from auth.users where email = 'validador1@example.com');

```

Pantalla de acceso: correo y contraseña con `supabase.auth.signInWithPassword(...)`; después lean `perfiles.rol` y, si no es validador, institución o admin, cierran la sesión con un mensaje.

Paso 1.9 R2 Secretos y esqueleto de la función notificar

1. En Supabase, *Edge Functions* → *Secrets*, agreguen `FIREBASE_SERVICE_ACCOUNT` (peguen el JSON completo de la cuenta de servicio) y `TELEGRAM_BOT_TOKEN`. Las variables `SUPABASE_URL` y `SUPABASE_SERVICE_ROLE_KEY` ya existen por defecto.
2. Creen y desplieguen la función con la CLI:

```

supabase login
supabase link --project-ref TU_REF # el REF está en la URL del proyecto
supabase functions new notificar
supabase functions deploy notificar
# si el despliegue pide Docker, prueben: supabase functions deploy notificar --use-api
# o creen la función desde el panel: Edge Functions > Deploy a new function > Via Editor

```

✓ Terminado cuando

Un `curl` a la URL de la función con la `service_role` key responde 200, y la función imprime en sus *Logs* el `project_id` de Firebase leído del secreto.

8.5 Fase 2 · Flujo principal (horas 6 a 16)

Objetivo: el recorrido completo de punta a punta. Alguien publica una alerta, el servidor encuentra los teléfonos cercanos y les llega una notificación con la distancia.

Paso 2.1 **R1** Registrar el dispositivo y consultar alertas cercanas

```
-- 004_funciones.sql
-- La app la llama al abrir y cada vez que cambia de celda. Reemplaza la celda anterior: sin historial.
create or replace function registrar_dispositivo(p_token text, p_plataforma text, p_celda text)
returns void language plpgsql security definer set search_path = public as $$
begin
    if auth.uid() is null then raise exception 'Sin sesión'; end if;
    insert into dispositivos (usuario_id, fcm_token, plataforma, celda)
    values (auth.uid(), p_token, p_plataforma, p_celda)
    on conflict (fcm_token) do update
        set usuario_id = excluded.usuario_id, celda = excluded.celda,
            activo = true, actualizado_en = now();
end $$;

-- Alertas para el mapa, buscadas por la CELDA del usuario (no por su ubicación exacta)
create or replace function alertas_cercanas(p_celda text, p_radio_m int default 25000)
returns table (id uuid, categoria categoria_alerta, nombre text, nivel smallint,
              estado estado_alerta, titulo text, descripcion text, referencia text,
              foto_path text, lat double precision, lon double precision,
              radio_actual_m int, creada_en timestampz, motivo_cierre text)
language sql stable security invoker set search_path = public, extensions as $$
select a.id, a.categoria, c.nombre, c.nivel, a.estado, a.titulo, a.descripcion, a.referencia,
       case when a.estado = 'resuelta' then null else a.foto_path end, -- resuelta: sin foto
       a.lat, a.lon, a.radio_actual_m, a.creada_en, a.motivo_cierre
from alertas a join categorias c on c.clave = a.categoria
where a.estado in ('no_confirmada', 'corroborada', 'verificada', 'resuelta')
and (a.estado <> 'resuelta' or a.cerrada_en > now() - interval '24 hours')
and st_dwithin(a.ubicacion,
              st_setsrid(st_pointfromgeohash(p_celda), 4326)::geography, p_radio_m)
order by c.nivel desc, a.creada_en desc
limit 100;
$$;
```

security invoker hace que la función respete las reglas RLS de quien la llama.

Paso 2.2 **R1** Crear reportes con todas las reglas

Esta función concentra las reglas de confianza: quién puede reportar, el límite por hora, la detección de duplicados y el estado inicial.

```
create or replace function crear_reporte(
    p_categoria categoria_alerta, p_titulo text, p_descripcion text, p_referencia text,
    p_lat double precision, p_lon double precision, p_foto_path text default null)
returns jsonb language plpgsql security definer set search_path = public, extensions as $$
declare
    v_uid      uuid := auth.uid();
    v_perfil   perfiles;
    v_cat      categorias;
    v_inst     boolean;
    v_dup      uuid;
    v_estado   estado_alerta;
    v_id       uuid;
begin
    -- 1. Solo cuentas verificadas (no anónimas) pueden reportar
    if v_uid is null or coalesce((auth.jwt() ->> 'is_anonymous')::boolean, false) then
        raise exception 'Verifica tu número de teléfono para poder reportar';
    end if;
    select * into v_perfil from perfiles where id = v_uid;
    select * into v_cat from categorias where clave = p_categoria;
    v_inst := v_perfil.rol in ('validador', 'institucion', 'admin');
```

```

if v_cat.solo_institucion and not v_inst then
  raise exception 'Esta categoría solo la pueden emitir instituciones';
end if;

-- 2. Límite: 3 reportes por hora
if not v_inst and (select count(*) from alertas
  where creada_por = v_uid and creada_en > now() - interval '1 hour') >= 3 then
  raise exception 'Alcanzaste el límite de reportes. Intenta más tarde';
end if;

-- 3. ¿Duplicado? Misma categoría, a menos de 500 m, en los últimos 30 minutos
select id into v_dup from alertas
  where categoria = p_categoria
    and estado in ('pendiente', 'no_confirmada', 'corroborada', 'verificada')
    and creada_en > now() - interval '30 minutes'
    and st_dwithin(ubicacion, st_setsrid(st_makepoint(p_lon, p_lat), 4326)::geography, 500)
order by creada_en desc limit 1;
if v_dup is not null then
  insert into confirmaciones (alerta_id, usuario_id, tipo)
  values (v_dup, v_uid, 'confirmo') on conflict do nothing;
  return jsonb_build_object('duplicada_de', v_dup);
end if;

-- 4. Estado inicial según quién reporta y qué categoría es
v_estado := case
  when v_inst then 'verificada'
  when v_cat.requiere_validacion or v_perfil.reputacion < -2 then 'pendiente'
  else 'no_confirmada' end;

insert into alertas (categoria, titulo, descripcion, referencia, foto_path, lat, lon, estado,
  creada_por, expira_en, publicada_en, verificada_por, verificada_en)
values (p_categoria, p_titulo, p_descripcion, p_referencia, p_foto_path, p_lat, p_lon, v_estado,
  v_uid, now() + v_cat.vigencia,
  case when v_estado <> 'pendiente' then now() end,
  case when v_inst then v_uid end,
  case when v_inst then now() end)
returning id into v_id;

return jsonb_build_object('alerta_id', v_id, 'estado', v_estado);
end $$;

```

Paso 2.3 **R1** Funciones geográficas: radio permitido y destinatarios

```

-- ¿Hasta dónde puede llegar esta alerta AHORA? (escalones + tope por confianza)
create or replace function radio_permitido(p_alerta uuid) returns int
language sql stable security definer set search_path = public, extensions as $$
select case
  when a.estado not in ('no_confirmada', 'corroborada', 'verificada') then 0
  when a.radio_manual_m is not null then a.radio_manual_m
  else least(
    (select coalesce(max(e.radio_m), 0) from escalones_radio e
      where e.categoria = a.categoria
      and e.minuto <= extract(epoch from now() - a.publicada_en) / 60
      * (select factor_tiempo from config)),
    case a.estado when 'no_confirmada' then c.radio_max_no_conf_m
      when 'corroborada' then 3000
      else 1000000 end)
  end
from alertas a join categorias c on c.clave = a.categoria
where a.id = p_alerta;
$$;

-- Teléfonos que deben recibirla: su celda o alguna de sus zonas está dentro del radio
-- (+700 m de margen de celda) y todavía no la han recibido
create or replace function dispositivos_objetivo(p_alerta uuid, p_radio_m int)
returns table (dispositivo_id uuid, fcm_token text, plataforma text)
language sql stable security definer set search_path = public, extensions as $$
select d.id, d.fcm_token, d.plataforma
from alertas a, dispositivos d

```

```

where a.id = p_alerta and d.activo
  and (st_dwithin(d.centro_celda, a.ubicacion, p_radio_m + 700)
    or exists (select 1 from zonas_usuario z
               where z.usuario_id = d.usuario_id
                  and st_dwithin(z.centro_celda, a.ubicacion, p_radio_m + 700)))
  and not exists (select 1 from entregas e
                  where e.alerta_id = a.id and e.dispositivo_id = d.id);
$$;

create or replace function dispositivos_validadores()
returns table (dispositivo_id uuid, fcm_token text, plataforma text)
language sql stable security definer set search_path = public as $$
  select d.id, d.fcm_token, d.plataforma
  from dispositivos d join perfiles p on p.id = d.usuario_id
  where d.activo and p.rol in ('validador', 'institucion', 'admin');
$$;

-- ¡IMPORTANTE! Estas funciones devuelven tokens: nadie desde la app debe poder llamarlas
revoke execute on function dispositivos_objetivo(uuid, int) from public, anon, authenticated;
revoke execute on function dispositivos_validadores()      from public, anon, authenticated;
revoke execute on function radio_permitido(uuid)           from public, anon, authenticated;
grant  execute on function dispositivos_objetivo(uuid, int) to service_role;
grant  execute on function dispositivos_validadores()      to service_role;
grant  execute on function radio_permitido(uuid)           to service_role;

```

⚠ Ojo

En Supabase, cualquier función del esquema public se puede llamar desde la app con la anon key (/rest/v1/rpc/...). Si olvidan el revoke, cualquiera podría descargar los tokens de todos los teléfonos. La prueba P14 (Fase 5) lo verifica.

Paso 2.4 R2 Edge Function notificar (completa)

Es el corazón del envío. La llaman el trigger de alertas (evento estado) y la tarea de pg_cron (evento ampliacion). Según el estado de la alerta: avisa del cierre, avisa a validadores, avisa una mejora de confianza o envía el anillo nuevo.

```

// supabase/functions/notificar/index.ts
import { createClient } from 'npm:@supabase/supabase-js@2';
import { JWT } from 'npm:google-auth-library@9';

const sb = createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!);
const sa = JSON.parse(Deno.env.get('FIREBASE_SERVICE_ACCOUNT')!);
const TG = Deno.env.get('TELEGRAM_BOT_TOKEN');

type Destino = { dispositivo_id: string; fcm_token: string; plataforma: string };

async function tokenDeAcceso(): Promise<string> { // OAuth2 para la API HTTP v1 de FCM
  const jwt = new JWT({ email: sa.client_email, key: sa.private_key,
    scopes: ['https://www.googleapis.com/auth/firebase.messaging'] });
  const { access_token } = await jwt.authorize();
  return access_token!;
}

async function enviarPush(acceso: string, d: Destino, datos: Record<string, string>) {
  const message: Record<string, unknown> = {
    token: d.fcm_token,
    data: datos, // Android: la app arma la notificación
    android: { priority: 'high', ttl: '3600s' },
  };
  if (d.plataforma === 'ios') { // iOS: notificación visible directa
    message.notification = { title: datos.titulo, body: datos.cuerpo };
    message.apns = { headers: { 'apns-priority': '10' }, payload: { aps: { sound: 'default' } } };
  }
  const r = await fetch(`https://fcm.googleapis.com/v1/projects/${sa.project_id}/messages:send`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${acceso}`, 'Content-Type': 'application/json' },
  });
}

```

```

    body: JSON.stringify({ message }),
  });
  if (r.status === 404) {
    // La app se desinstaló: token inválido
    await sb.from('dispositivos').update({ activo: false }).eq('id', d.dispositivo_id);
  }
  return r.ok;
}

async function enviarATodos(acceso: string, destinos: Destino[], datos: Record<string, string>) {
  let ok = 0;
  for (let i = 0; i < destinos.length; i += 100) {
    // de 100 en 100, en paralelo
    const lote = destinos.slice(i, i + 100);
    ok += (await Promise.all(lote.map((d) => enviarPush(acceso, d, datos)))).filter(Boolean).length;
  }
  return ok;
}

async function yaLaRecibieron(alertaId: string): Promise<Destino[]> {
  const { data } = await sb.from('entregas')
    .select('dispositivo_id, dispositivos(fcm_token, plataforma, activo)').eq('alerta_id', alertaId);
  return (data ?? []).filter((e: any) => e.dispositivos?.activo)
    .map((e: any) => ({ dispositivo_id: e.dispositivo_id, ...e.dispositivos }));
}

Deno.serve(async (req) => {
  const { alerta_id, evento, anterior } = await req.json();
  const { data: a } = await sb.from('alertas')
    .select('*', categorias(nombre, nivel)).eq('id', alerta_id).single();
  if (!a) return new Response('La alerta no existe', { status: 404 });

  const acceso = await tokenDeAcceso();
  const datos: Record<string, string> = {
    // FCM exige que todos los valores sean texto
    alerta_id: a.id, categoria: a.categoria, nivel: String(a.categorias.nivel), estado: a.estado,
    titulo: a.categorias.nombre.toUpperCase(), cuerpo: a.titulo,
    lat: String(a.lat), lon: String(a.lon), foto: a.foto_path ? '1' : '0',
  };

  // 1) Cierre o descarte: avisar a todos los que ya la tenían
  if (a.estado === 'resuelta' || a.estado === 'descartada') {
    datos.tipo = 'cierre';
    datos.titulo = a.estado === 'resuelta' ? 'RESUELTA' : 'DESCARTADA';
    datos.cuerpo = a.estado === 'resuelta'
      ? `${a.motivo_cierre} ?? 'El caso fue resuelto'. Gracias por tu ayuda.`
      : 'La información no pudo confirmarse.';
    return Response.json({ cierre: await enviarATodos(acceso, await yaLaRecibieron(a.id), datos) });
  }

  // 2) En revisión: avisar solo a los validadores
  if (a.estado === 'pendiente') {
    const { data: val } = await sb.rpc('dispositivos_validadores');
    datos.tipo = 'validacion';
    return Response.json({ validadores: await enviarATodos(acceso, val ?? [], datos) });
  }

  // 3) Subió de confianza (p. ej. ahora está VERIFICADA): avisar a quienes ya la tenían
  let actualizados = 0;
  if (evento === 'estado' && ['corroborada', 'verificada'].includes(a.estado)
    && ['no_confirmada', 'corroborada'].includes(anterior)) {
    actualizados = await enviarATodos(acceso, await yaLaRecibieron(a.id),
      { ...datos, tipo: 'actualizacion', titulo: `AHORA ${a.estado.toUpperCase()}` });
  }

  // 4) Anillo nuevo: teléfonos dentro del radio permitido que aún no la reciben
  const { data: radio } = await sb.rpc('radio_permitido', { p_alerta: a.id });
  if (!radio || radio <= 0) return Response.json({ actualizados });
  if (radio > a.radio_actual_m) {
    await sb.from('alertas').update({ radio_actual_m: radio }).eq('id', a.id);
  }
  const { data: candidatos } = await sb.rpc('dispositivos_objetivo',
    { p_alerta: a.id, p_radio_m: radio });

```

```

let enviados = 0;
if (candidatos?.length) {
  // "Apartamos" cada envío en entregas ANTES de mandarlo: si dos ejecuciones coinciden,
  // solo una logra insertar la fila, y solo esa envía. Así nadie la recibe dos veces.
  const { data: apartados } = await sb.from('entregas')
    .upsert(candidatos.map((d: Destino) =>
      ({ alerta_id: a.id, dispositivo_id: d.dispositivo_id, radio_m: radio })),
      { onConflict: 'alerta_id,dispositivo_id', ignoreDuplicates: true })
    .select('dispositivo_id');
  const ids = new Set((apartados ?? []).map((e: any) => e.dispositivo_id));
  enviados = await enviarATodos(acceso, candidatos.filter((d: Destino) => ids.has(d.dispositivo_id)),
    { ...datos, tipo: 'nueva', radio_m: String(radio) });
}
const telegram = await enviarTelegram(a, radio); // Paso 4.1
return Response.json({ radio, enviados, actualizados, telegram });
});

async function enviarTelegram(a: any, radio: number) {
  if (!TG) return 0;
  const { data: chats } = await sb.rpc('telegram_objetivo', { p_alerta: a.id, p_radio_m: radio });
  let n = 0;
  for (const { chat_id } of chats ?? []) {
    const texto = `${a.categorias.nombre.toUpperCase()} (${a.estado.replace('_', ' ')})\n`
      + `${a.titulo}\n${a.descripcion ?? ''}\n`
      + `Mapa: https://www.openstreetmap.org/?mlat=${a.lat}&mlon=${a.lon}#map=16/${a.lat}/${a.lon}\n`
      + 'No sustituye al 911. En una emergencia, llama al 911.';
    const r = await fetch(`https://api.telegram.org/bot${TG}/sendMessage`, {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ chat_id, text: texto }) });
    if (r.ok) { n++; await sb.from('entregas_telegram').insert({ alerta_id: a.id, chat_id }); }
  }
  return n;
}

```

Hasta el Paso 4.1, la función telegram_objetivo todavía no existe: dejen TELEGRAM_BOT_TOKEN sin configurar y enviarTelegram devolverá 0. Desplieguen con supabase functions deploy notificar.

Paso 2.5 R1 Disparar notificar automáticamente (pg_net + Vault)

```

-- 005_disparadores.sql
-- 1) Guardar la URL de las funciones y la service_role key en Vault (una sola vez)
select vault.create_secret('https://TU_REF.supabase.co/functions/v1', 'url_funciones');
select vault.create_secret('TU_SERVICE_ROLE_KEY', 'service_role_key');

-- 2) Función auxiliar para llamar a una Edge Function desde SQL
create or replace function llamar_funcion(p_funcion text, p_cuerpo jsonb) returns void
language plpgsql security definer set search_path = public as $$
begin
  perform net.http_post(
    url      := (select decrypted_secret from vault.decrypted_secrets where name = 'url_funciones')
              || '/' || p_funcion,
    headers  := jsonb_build_object(
      'Content-Type', 'application/json',
      'Authorization', 'Bearer ' || (select decrypted_secret from vault.decrypted_secrets
                                    where name = 'service_role_key')),
    body     := p_cuerpo);
end $$;
revoke execute on function llamar_funcion(text, jsonb) from public, anon, authenticated;

-- 3) Cada vez que se crea una alerta o cambia su estado, se llama a notificar
create or replace function al_cambiar_estado() returns trigger
language plpgsql security definer set search_path = public as $$
begin
  perform llamar_funcion('notificar', jsonb_build_object(
    'alerta_id', new.id, 'evento', 'estado',
    'anterior', case when tg_op = 'UPDATE' then old.estado::text end));
  return new;
end $$;

```

```
create trigger alertas_estado_insert after insert on alertas
  for each row execute function al_cambiar_estado();
create trigger alertas_estado_update after update of estado on alertas
  for each row when (old.estado is distinct from new.estado)
  execute function al_cambiar_estado();
```

pg_net envía la petición **después** de que la transacción se confirma, así que la función ya encuentra la alerta guardada.

Paso 2.6 R4 Geohash y ubicación en la app

```
// Lib/nucleo/geohash.dart: mismo algoritmo que PostGIS (geohash(17.9581, -102.1942) == '9epq4t')
const _base32 = '0123456789bcdefghjkmnpqrstvwxyz';
```

```
String geohash(double lat, double lon, [int precision = 6]) {
  var latMin = -90.0, latMax = 90.0, lonMin = -180.0, lonMax = 180.0;
  final sb = StringBuffer();
  var bit = 0, ch = 0;
  var esLon = true;
  while (sb.length < precision) {
    if (esLon) {
      final mid = (lonMin + lonMax) / 2;
      if (lon >= mid) { ch = (ch << 1) | 1; lonMin = mid; } else { ch = ch << 1; lonMax = mid; }
    } else {
      final mid = (latMin + latMax) / 2;
      if (lat >= mid) { ch = (ch << 1) | 1; latMin = mid; } else { ch = ch << 1; latMax = mid; }
    }
    esLon = !esLon;
    if (++bit == 5) { sb.write(_base32[ch]); bit = 0; ch = 0; }
  }
  return sb.toString();
}
```

```
// Lib/nucleo/ubicacion.dart
class Ubicacion {
  /// Pide permiso "mientras se usa la app". Devuelve true si se concedió.
  static Future<bool> pedirPermiso() async {
    var p = await Geolocator.checkPermission();
    if (p == LocationPermission.denied) p = await Geolocator.requestPermission();
    return p == LocationPermission.whileInUse || p == LocationPermission.always;
  }

  /// Lee la posición, la guarda SOLO en el teléfono y sube la celda si cambió.
  static Future<void> actualizarMiCelda({bool forzar = false}) async {
    final prefs = await SharedPreferences.getInstance();
    double lat, lon;
    final demoLat = prefs.getDouble('demo_lat'); // modo demo (Paso 3.7)
    if (demoLat != null) {
      lat = demoLat;
      lon = prefs.getDouble('demo_lon')!;
    } else {
      final pos = await Geolocator.getCurrentPosition(
        locationSettings: const LocationSettings(accuracy: LocationAccuracy.medium));
      lat = pos.latitude;
      lon = pos.longitude;
    }
    await prefs.setDouble('mi_lat', lat); // se queda en el teléfono
    await prefs.setDouble('mi_lon', lon);

    final celda = geohash(lat, lon, 6);
    if (!forzar && prefs.getString('mi_celda') == celda) return; // no cambió: no se envía nada
    final token = await FirebaseMessaging.instance.getToken();
    if (token == null) return;
    await Supabase.instance.client.rpc('registrar_dispositivo', params: {
      'p_token': token,
      'p_plataforma': Platform.isIOS ? 'ios' : 'android',
      'p_celda': celda,
    });
    await prefs.setString('mi_celda', celda);
  }
}
```

}

- **Cuándo llamar a** actualizarMiCelda(): al terminar la bienvenida (después de los permisos), cada vez que la app vuelve al frente (WidgetsBindingObserver, estado resumed) y, con la app abierta, con Geolocator.getPositionStream y distanceFilter: 300.
- **Precisión media** (wifi y antenas) es suficiente para celdas de 1 km y gasta mucha menos batería que el GPS.
- **Prueba unitaria:** expect(geohash(17.9581, -102.1942), '9epq4t');

Paso 2.7 R4 Recibir y mostrar las notificaciones

```
// Lib/nucleo/notificaciones.dart
final _plugin = FlutterLocalNotificationsPlugin();
const _ajustes = InitializationSettings(android: AndroidInitializationSettings('@mipmap/ic_launcher'));

const _canales = {
  4: AndroidNotificationChannel('nivel4', 'Alertas críticas',
    description: 'Menores desaparecidos, evacuaciones', importance: Importance.max),
  3: AndroidNotificationChannel('nivel3', 'Alertas altas', importance: Importance.high),
  2: AndroidNotificationChannel('nivel2', 'Alertas', importance: Importance.defaultImportance),
  1: AndroidNotificationChannel('nivel1', 'Informativas', importance: Importance.low),
};

const _colores = {4: Color(0xFFD62828), 3: Color(0xFFFF77F0), 2: Color(0xFFE09F1F), 1: Color(0xFF3A86FF)};
const _estados = {'no_confirmada': 'No confirmada', 'corroborada': 'Corroborada', 'verificada': 'Verificada'};

class Notificaciones {
  static Future<void> inicializar({void Function(String alertaId)? alTocar}) async {
    await _plugin.initialize(_ajustes, onDidReceiveNotificationResponse: (r) {
      if (r.payload != null) alTocar?.call(r.payload!);
    });
    final android = _plugin.resolvePlatformSpecificImplementation<
      AndroidFlutterLocalNotificationsPlugin>();
    for (final c in _canales.values) {
      await android?.createNotificationChannel(c);
    }
    await FirebaseMessaging.instance.requestPermission(); // Android 13+ e iOS
    FirebaseMessaging.onMessage.listen((m) => procesarAlerta(m.data)); // con la app abierta
  }
}

/// Con la app cerrada o en segundo plano. Debe ser una función de primer nivel.
@pragma('vm:entry-point')
Future<void> alRecibirEnSegundoPlano(RemoteMessage m) async {
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  await _plugin.initialize(_ajustes);
  await procesarAlerta(m.data);
}

Future<void> procesarAlerta(Map<String, dynamic> d) async {
  final prefs = await SharedPreferences.getInstance();
  var nivel = int.tryParse(d['nivel'] ?? '') ?? 2;
  var cuerpo = (d['cuerpo'] ?? '') as String;

  if (d['tipo'] == 'nueva') {
    // Distancia exacta calculada EN EL TELÉFONO: a mí o a mis zonas guardadas (Paso 3.7)
    final puntos = <List<double>>[
      if (prefs.getDouble('mi_lat') != null) [prefs.getDouble('mi_lat')!, prefs.getDouble('mi_lon')!],
      for (final z in prefs.getStringList('zonas') ?? <String>[])
        (jsonDecode(z) as List).map((e) => (e as num).toDouble()).toList(),
    ];
    if (puntos.isNotEmpty) {
      final aLat = double.parse(d['lat']), aLon = double.parse(d['lon']);
      final metros = puntos
        .map((p) => Geolocator.distanceBetween(p[0], p[1], aLat, aLon)).reduce(min);
      if (metros > double.parse(d['radio_m'] ?? '0')) nivel = min(nivel, 2); // margen: sin alarma
      final dist = metros < 1000
        ? '${(metros / 10).round() * 10} m'
        : '${(metros / 1000).toStringAsFixed(1)} km';
    }
  }
}
```

```

        cuerpo = 'A $dist de ti · ${_estados[d['estado']] ?? ''} · $cuerpo';
    }
}
final canal = _canales[nivel!];
await _plugin.show(
  d['alerta_id'].hashCode, d['titulo'], cuerpo,
  NotificationDetails(android: AndroidNotificationDetails(canal.id, canal.name,
    importance: canal.importance, priority: Priority.high, color: _colores[nivel],
    styleInformation: BigTextStyleInformation(cuerpo))),
  payload: d['alerta_id'],
);
}

```

⚠ Ojo

Los canales de Android se crean una sola vez: si cambian la importancia de un canal, desinstalen la app del emulador para verlo. Y la notificación de tipo validacion solo les llega a los validadores: si R5 instala la app con su cuenta de validador, también la recibe.

Paso 2.8 R3 Pantallas: Inicio, Detalle y Reportar

Mientras R1 y R2 terminan el backend, R3 puede construir las pantallas con una lista de alertas falsas en memoria (mismo formato que alertas_cercanas) y después cambiar a la llamada real.

Pantalla	Elementos	Datos
Inicio	Barra con "Recibes alertas: aquí + N zonas"; mapa con círculos del radio actual y marcadores por nivel; lista ordenada por nivel y distancia; botón rojo flotante "Reportar"; aviso fijo "No sustituye al 911".	rpc('alertas_cercanas', {p_celda})
Detalle	Encabezado del color del nivel; insignia de estado; foto (enlace firmado de 10 min); distancia y hace cuánto; descripción y referencia; quién la validó; botones "Yo también lo vi", "Ya no está", "Parece falsa", "Llamar al 911", "Compartir"; si soy el autor: "Marcar como resuelta".	storage.from('fotos').createSignedUrl(path, 600)
Reportar	Paso 1: cuadrícula de categorías (las grises son solo para instituciones). Paso 2: mapa con pin arrastrable (empieza en mi ubicación) y campo "referencia". Paso 3: título, descripción, foto opcional, casilla "Confirmando que la información es verdadera" y, para personas, "Tengo el consentimiento del familiar o tutor".	rpc('crear_reporte', ...)

```

// Mapa de inicio (fragmento) con flutter_map y OpenStreetMap
FlutterMap(
  options: MapOptions(initialCenter: LatLng(miLat, miLon), initialZoom: 14),
  children: [
    TileLayer(
      urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
      userAgentPackageName: 'mx.alertacerca.alerta_cerca', // Lo exige la política de OSM
    ),
    CircleLayer(circles: [
      for (final a in alertas)
        CircleMarker(point: LatLng(a.lat, a.lon), radius: a.radioActualM.toDouble(),
          useRadiusInMeter: true, color: colorNivel(a.nivel).withAlpha(30),
          borderColor: colorNivel(a.nivel), borderStrokeWidth: 1.5),
    ]),
    MarkerLayer(markers: [
      for (final a in alertas)
        Marker(point: LatLng(a.lat, a.lon), width: 36, height: 36,
          child: GestureDetector(onTap: () => abrirDetalle(a), child: IconoCategoria(a))),
    ]),
    const RichAttributionWidget(attributions: [TextSourceAttribution('@ OpenStreetMap')]),
  ],
)

// Enviar un reporte (fragmento de La pantalla Reportar)
Future<void> enviarReporte() async {

```

```

final sb = Supabase.instance.client;
String? fotoPath;
if (foto != null) {
  // Recomprimir quita los metadatos EXIF (GPS, modelo del teléfono) antes de subir
  final bytes = await FlutterImageCompress.compressWithFile(foto!.path,
    minWidth: 1280, minHeight: 1280, quality: 75, keepExif: false);
  fotoPath = '${sb.auth.currentUser!.id}/${DateTime.now().millisecondsSinceEpoch}.jpg';
  await sb.storage.from('fotos').uploadBinary(fotoPath, bytes!,
    fileOptions: const FileOptions(contentType: 'image/jpeg'));
}
try {
  final r = await sb.rpc('crear_reporte', params: {
    'p_categoria': categoria, 'p_titulo': titulo, 'p_descripcion': descripcion,
    'p_referencia': referencia, 'p_lat': pin.latitude, 'p_lon': pin.longitude,
    'p_foto_path': fotoPath,
  });
  if (r['duplicada_de'] != null) {
    mostrarMensaje('Ya había un reporte igual cerca. Sumamos tu confirmación.');
  } else if (r['estado'] == 'pendiente') {
    mostrarMensaje('Tu reporte está en revisión. Un validador lo revisará en minutos.');
  } else {
    mostrarMensaje('Reporte publicado. Avisamos a las personas cercanas.');
  }
} on PostgreSQLException catch (e) {
  mostrarMensaje(e.message); // p. ej. "Alcanzaste el Límite de reportes..."
}
}

```

Paso 2.9 **Todos** Prueba integrada #1: la primera alerta de punta a punta

1. R4 abre la app en el emulador, acepta los permisos y coloca la ubicación del emulador cerca del centro de Lázaro Cárdenas (en *Extended controls* → *Location*, o con el comando de abajo).
2. R1 revisa que exista una fila en dispositivos con su celda de 6 caracteres (y sin coordenadas).
3. R1 publica una alerta verificada de prueba desde el SQL Editor (ahí no hay usuario, por eso se inserta directo).
4. En pocos segundos el emulador muestra la notificación "INCENDIO · A 300 m de ti · Verificada · PRUEBA...".
5. Si no llega: revisen los Logs de la Edge Function y las respuestas de pg_net (consultas de abajo).

```

adb emu geo fix -102.1942 17.9608 # jojo! primero Longitud, luego Latitud

insert into alertas (categoria, titulo, descripcion, lat, lon, estado, creada_por,
  expira_en, publicada_en)
values ('incendio', 'PRUEBA: humo en bodega', 'Prueba del equipo', 17.9581, -102.1942, 'verificada',
  (select id from perfiles limit 1), now() + interval '1 hour', now());

select * from entregas order by enviada_en desc limit 5; -- ¿se registró el envío?
select id, status_code, content::text from net._http_response
order by created desc limit 5; -- ¿qué respondió notificar?

```

✓ Terminado cuando

Una alerta insertada en la base de datos llega como notificación al emulador en menos de 10 segundos, con la distancia correcta, y queda registrada en entregas.

8.6 Fase 3 · Inteligencia y confianza (horas 16 a 26)

Objetivo: el radio crece solo, los vecinos pueden confirmar, los validadores trabajan desde el panel y las alertas se cierran con aviso.

Paso 3.1 **R1** Radio dinámico con pg_cron

```
-- 006_tareas.sql
-- Cada 15 segundos: ¿alguna alerta ya puede llegar más lejos? Entonces se notifica el anillo nuevo
select cron.schedule('ampliar-radios', '15 seconds', $$
  select llamar_funcion('notificar', jsonb_build_object('alerta_id', a.id, 'evento', 'ampliacion'))
  from alertas a
  where a.estado in ('no_confirmada', 'corroborada', 'verificada')
  and radio_permitido(a.id) > a.radio_actual_m;
$$);

-- Para la demo (¡regresarlo a 1 al terminar!)
update config set factor_tiempo = 30;

-- ¿Está corriendo?
select jobid, status, start_time, return_message
from cron.job_run_details order by start_time desc limit 10;
```

Si una ejecución coincide con otra, no pasa nada: la tabla entregará "aparta" cada envío (Paso 2.4) y nadie recibe la alerta dos veces.

Paso 3.2 **R1** Confirmaciones de vecinos y corroboración automática

```
create or replace function confirmar_alerta(p_alerta uuid, p_tipo text) returns void
language plpgsql security definer set search_path = public as $$
begin
  if auth.uid() is null or coalesce((auth.jwt() ->> 'is_anonymous')::boolean, false) then
    raise exception 'Verifica tu número para confirmar alertas';
  end if;
  if exists (select 1 from alertas where id = p_alerta and creada_por = auth.uid()) then
    raise exception 'No puedes confirmar tu propio reporte';
  end if;
  insert into confirmaciones (alerta_id, usuario_id, tipo) values (p_alerta, auth.uid(), p_tipo)
  on conflict (alerta_id, usuario_id) do update set tipo = excluded.tipo, creada_en = now();
end $$;

create or replace function revisar_confirmaciones() returns trigger
language plpgsql security definer set search_path = public as $$
declare v_si int; v_falsa int;
begin
  select count(*) filter (where tipo = 'confirmando'), count(*) filter (where tipo = 'parece_falsa')
  into v_si, v_falsa
  from confirmaciones where alerta_id = new.alerta_id;

  if v_si >= 3 then
    -- 3 vecinos verificados la confirman
    update alertas set estado = 'corroborada' where id = new.alerta_id and estado = 'no_confirmada';
  end if;
  if v_falsa >= 3 then
    -- 3 dicen que parece falsa: regresa a revisión
    update alertas set estado = 'pendiente'
    where id = new.alerta_id and estado in ('no_confirmada', 'corroborada');
  end if;
  return new;
end $$;
create trigger confirmaciones_revisar after insert or update on confirmaciones
for each row execute function revisar_confirmaciones();
```

Paso 3.3 **R1** Acciones de los validadores y bitácora

```
create or replace function validar_alerta(p_alerta uuid, p_accion text,
                                          p_motivo text default null, p_radio_m int default null)
returns void language plpgsql security definer set search_path = public as $$
declare v_alerta alertas;
begin
  select * into v_alerta from alertas where id = p_alerta for update;
  if not found then raise exception 'La alerta no existe'; end if;
  -- El autor solo puede marcar SU alerta como resuelta; todo lo demás es de validadores
  if not es_validador() and not (p_accion = 'resolver' and v_alerta.creada_por = auth.uid()) then
    raise exception 'No autorizado';
  end if;
```

```

case p_accion
  when 'verificar' then
    update alertas set estado = 'verificada', verificada_por = auth.uid(), verificada_en = now(),
                      publicada_en = coalesce(publicada_en, now())
    where id = p_alerta;
    update perfiles set reputacion = reputacion + 1 where id = v_alerta.creada_por;
  when 'descartar' then
    update alertas set estado = 'descartada', cerrada_en = now(), motivo_cierre = p_motivo
    where id = p_alerta;
    update perfiles set reputacion = reputacion - 2 where id = v_alerta.creada_por;
  when 'resolver' then
    update alertas set estado = 'resuelta', cerrada_en = now(), motivo_cierre = p_motivo
    where id = p_alerta;
  when 'ajustar_radio' then
    update alertas set radio_manual_m = p_radio_m where id = p_alerta;    -- pg_cron lo envía
  else
    raise exception 'Acción desconocida: %', p_accion;
end case;

insert into bitacora (alerta_id, usuario_id, accion, detalle)
values (p_alerta, auth.uid(), p_accion, jsonb_build_object('motivo', p_motivo, 'radio_m', p_radio_m));
end $$;

```

Paso 3.4 R1 Expiración y limpieza automáticas

```

select cron.schedule('expirar-alertas', '*/5 * * * *', $$
  update alertas set estado = 'expirada', cerrada_en = now()
  where estado in ('pendiente', 'no_confirmada', 'corroborada', 'verificada') and expira_en < now();
$$);

select cron.schedule('limpieza-diaria', '0 4 * * *', $$
  delete from entregas          where enviada_en < now() - interval '30 days';
  delete from entregas_telegram where enviada_en < now() - interval '30 days';
  update dispositivos set activo = false where actualizado_en < now() - interval '60 days';
$$);

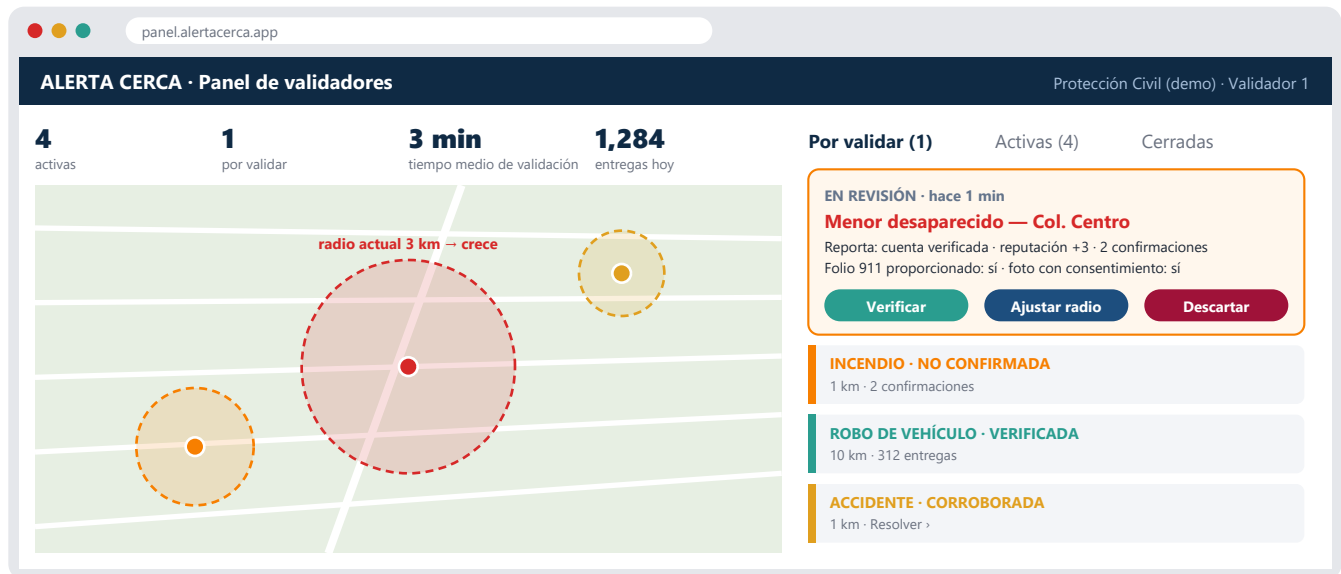
```

La hora de pg_cron está en UTC: las 4:00 UTC son las 22:00 en Lázaro Cárdenas (UTC-6).

Paso 3.5 R2 Probar todas las ramas de notificar

Acción	Respuesta esperada de notificar (Logs o net_http_response)
Ciudadano reporta "menor desaparecido"	{"validadores": 1}: solo le llega al validador
El validador lo verifica	{"radio": 1000, "enviados": N, ...}
pg_cron amplía el radio	{"radio": 3000, "enviados": M} (M = solo el anillo nuevo)
Tres vecinos confirman un incendio no confirmado	{"actualizados": N, ...} (llega "AHORA CORROBORADA")
El validador lo resuelve	{"cierre": N}: todos los que la tenían reciben "RESUELTA"
Token de una app desinstalada	El dispositivo queda con activo = false

Paso 3.6 R5 Panel de validadores completo



- **Encabezado con métricas:** activas, por validar, tiempo medio de validación y entregas de hoy (vista métricas, Paso 4.3).
- **Mapa** con el radio actual de cada alerta, coloreado por estado.
- **Pestañas:** Por validar (pendientes, ordenadas por nivel y antigüedad), Activas y Cerradas.
- **Detalle:** foto, descripción, reputación del autor, número de confirmaciones y "parece falsa", y bitácora.
- **Acciones:** Verificar; Ajustar radio (1, 3, 5, 10 o 25 km); Descartar (motivo obligatorio); Resolver (motivo, p. ej. "Menor localizado sano y salvo").
- **Emitir alerta oficial:** el mismo formulario de reporte; como lo emite un validador, sale verificada.

```
// Lista en vivo: se actualiza sola cuando entra un reporte (Realtime respeta Las reglas RLS)
final flujo = Supabase.instance.client
  .from('alertas')
  .stream(primaryKey: ['id'])
  .order('creada_en', ascending: false)
  .limit(200);

StreamBuilder<List<Map<String, dynamic>>>(
  stream: flujo,
  builder: (context, snap) {
    final filas = snap.data ?? [];
    final porValidar = filas.where((a) => a['estado'] == 'pendiente').toList();
    final activas = filas.where((a) =>
      ['no_confirmada', 'corroborada', 'verificada'].contains(a['estado'])).toList();
    return PanelConPestañas(porValidar: porValidar, activas: activas);
  },
);

// Botones de acción
Future<void> validar(String id, String accion, {String? motivo, int? radio}) =>
  Supabase.instance.client.rpc('validar_alerta', params: {
    'p_alerta': id, 'p_accion': accion, 'p_motivo': motivo, 'p_radio_m': radio,
  });
```

Paso 3.7 R4 Mis zonas, modo demo y cuenta con teléfono

```
// Guardar una zona (pulsación larga en el mapa de "Mis zonas"; máximo 3)
Future<void> guardarZona(String nombre, LatLng p) async {
  final sb = Supabase.instance.client;
  await sb.from('zonas_usuario').insert({
    'usuario_id': sb.auth.currentUser!.id,
    'nombre': nombre,
    'celda': geohash(p.latitude, p.longitude, 6), // al servidor: solo la celda
  });
  final prefs = await SharedPreferences.getInstance(); // en el teléfono: el punto exacto
  final zonas = prefs.getStringList('zonas') ?? <String>[];
  zonas.add(jsonEncode([p.latitude, p.longitude]));
  await prefs.setStringList('zonas', zonas);
}

// Modo demo: solo en compilación de depuración (kDebugMode). Ajustes > Ubicación simulada
const puntosDemo = {
  'A · 300 m': [17.9608, -102.1942],
  'B · 2.6 km': [17.9815, -102.1942],
  'C · 6 km': [18.0121, -102.1942],
  'D · Zihuatanejo': [17.6417, -101.5517],
};

Future<void> usarPuntoDemo(List<double> p) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setDouble('demo_lat', p[0]);
  await prefs.setDouble('demo_lon', p[1]);
  await Ubicacion.actualizarMiCelda(forzar: true);
}

// Convertir la cuenta anónima en verificada SIN perder su id (ni sus zonas)
Future<void> enviarCodigo(String numero) =>
  Supabase.instance.client.auth.updateUser(UserAttributes(phone: '+52$numero'));

Future<void> verificarCodigo(String numero, String codigo) =>
  Supabase.instance.client.auth.verifyOTP(
    type: OtpType.phoneChange, phone: '+52$numero', token: codigo);
```

La app pide el número solo cuando la persona quiere reportar o confirmar, nunca para recibir alertas.

Paso 3.8 R3 Confianza en la interfaz, bienvenida y permisos

- **Insignias** en la lista, el mapa y el detalle con los colores de la sección 4.4.
- **Botones de confirmación:** llaman a `confirmar_alerta`; si la cuenta es anónima, abren la pantalla de verificación de teléfono.
- **"Marcar como resuelta"** (solo el autor): llama a `validar_alerta` con `resolver` y un motivo.
- **Bienvenida** en 3 pantallas: "Si algo importante pasa cerca de ti, te avisamos" · "Tu ubicación exacta nunca sale de tu teléfono; solo usamos una zona de ~1 km" · "No sustituye al 911".
- **Permisos con explicación previa:** primero una pantalla propia que dice para qué sirve el permiso y después el cuadro del sistema. La ubicación "todo el tiempo" se ofrece como opcional.

```
// Compartir con contexto: fecha, hora y estado, para que no circule como si fuera oficial
final texto = '[ALERTA CERCA] ${a.nombre.toUpperCase()} (${estadoLegible(a.estado)})\n'
  '${DateFormat('dd/MM HH:mm').format(a.creadaEn.toLocal())} · ${a.titulo}\n'
  '${a.referencia ?? ''}\n'
  'Mapa: https://www.openstreetmap.org/?mlat=${a.lat}&mlon=${a.lon}#map=17/${a.lat}/${a.lon}\n'
  'Verifica antes de reenviar. En una emergencia, llama al 911.';
await Share.share(texto); // en versiones nuevas: SharePlus.instance.share(ShareParams(text: texto))
```

Paso 3.9 Todos Prueba integrada #2: el escenario completo de la demo

1. Cuatro dispositivos (teléfonos o emuladores) con la app y el modo demo en los puntos A, B, C y D.
2. R1 ejecuta `update config set factor_tiempo = 30;`
3. Una cuenta ciudadana verificada reporta "menor desaparecido" en el punto del suceso: queda **en revisión** y solo le llega al validador.
4. El validador lo verifica en el panel: A suena al instante.
5. Unos 30 segundos después llega a B; unos 2 minutos después, a C. D nunca la recibe.
6. El validador lo resuelve: A, B y C reciben "RESUELTA".
7. Revisen que nadie recibió la alerta dos veces (entregas tiene una fila por dispositivo).

✓ Terminado cuando

El escenario completo corre tres veces seguidas sin fallar, con tiempos parecidos a los esperados. Ese es el momento de pasar a la Fase 4.

8.7 Fase 4 · Canales alternos e integración (horas 26 a 34)

Paso 4.1 R2 R1 Bot de Telegram

R1 crea la función que elige a los suscriptores cercanos:

```
create or replace function telegram_objetivo(p_alerta uuid, p_radio_m int)
returns table (chat_id bigint)
language sql stable security definer set search_path = public, extensions as $$
  select s.chat_id from alertas a, suscriptores_telegram s
  where a.id = p_alerta and s.activo
        and st_dwithin(s.centro_celda, a.ubicacion, p_radio_m + 700)
        and not exists (select 1 from entregas_telegram e
                        where e.alerta_id = a.id and e.chat_id = s.chat_id);
$$;
revoke execute on function telegram_objetivo(uuid, int) from public, anon, authenticated;
grant  execute on function telegram_objetivo(uuid, int) to service_role;
```

R2 crea el geohash compartido (el mismo algoritmo que la app) y la función que recibe los mensajes del bot:

```
// supabase/functions/_compartido/geohash.ts
const B32 = '0123456789bcdefghjkmnpqrstuvwxyz';
export function geohash(lat: number, lon: number, precision = 6): string {
  let [latMin, latMax, lonMin, lonMax] = [-90, 90, -180, 180];
  let s = '', bit = 0, ch = 0, esLon = true;
  while (s.length < precision) {
    if (esLon) {
      const mid = (lonMin + lonMax) / 2;
      if (lon >= mid) { ch = (ch << 1) | 1; lonMin = mid; } else { ch <= 1; lonMax = mid; }
    } else {
      const mid = (latMin + latMax) / 2;
      if (lat >= mid) { ch = (ch << 1) | 1; latMin = mid; } else { ch <= 1; latMax = mid; }
    }
    esLon = !esLon;
    if (++bit === 5) { s += B32[ch]; bit = 0; ch = 0; }
  }
  return s;
}

// supabase/functions/telegram-webhook/index.ts (desplegar con --no-verify-jwt)
import { createClient } from 'npm:@supabase/supabase-js@2';
import { geohash } from '../_compartido/geohash.ts';

const TOKEN = Deno.env.get('TELEGRAM_BOT_TOKEN')!;
```

```

const SECRETO = Deno.env.get('TELEGRAM_SECRETO')!;
const sb = createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!);

const responder = (chat_id: number, text: string, reply_markup?: unknown) =>
  fetch('https://api.telegram.org/bot${TOKEN}/sendMessage', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ chat_id, text, reply_markup }) });

Deno.serve(async (req) => {
  // Telegram manda este encabezado con el secreto que registramos en setWebhook
  if (req.headers.get('X-Telegram-Bot-Api-Secret-Token') !== SECRETO) {
    return new Response('No autorizado', { status: 401 });
  }
  const { message } = await req.json();
  if (!message) return new Response('ok');
  const chat = message.chat.id;

  if (message.location) {
    const celda = geohash(message.location.latitude, message.location.longitude, 6);
    await sb.from('suscriptores_telegram').upsert({ chat_id: chat, celda, activo: true });
    await responder(chat, 'Listo. Te avisaré de alertas cerca de esa zona. Solo guardamos una zona '
      + 'de ~1 km, no tu ubicación exacta. Escribe /baja para dejar de recibirlas.',
      { remove_keyboard: true });
  } else if (message.text === '/baja') {
    await sb.from('suscriptores_telegram').update({ activo: false }).eq('chat_id', chat);
    await responder(chat, 'Te diste de baja. Escribe /start para volver.');
```

```

  } else {
    await responder(chat, 'Hola, soy ALERTA CERCA. Comparte tu ubicación para recibir alertas de '
      + 'emergencia de tu zona. No sustituye al 911.',
      { keyboard: [[{ text: 'Compartir mi ubicación', request_location: true }]],
        resize_keyboard: true, one_time_keyboard: true });
  }
  return new Response('ok');
});

supabase secrets set TELEGRAM_SECRETO=una_cadena_larga_y_aleatoria_123
supabase secrets set TELEGRAM_BOT_TOKEN=EL_TOKEN_DE_BOTFATHER
supabase functions deploy telegram-webhook --no-verify-jwt # Telegram no manda JWT de Supabase
supabase functions deploy notificar # para que lea el token nuevo

# Registrar el webhook (una sola vez). Reemplacen <TOKEN> y TU_REF
curl "https://api.telegram.org/bot<TOKEN>/setWebhook" \
  -d "url=https://TU_REF.supabase.co/functions/v1/telegram-webhook" \
  -d "secret_token=una_cadena_larga_y_aleatoria_123"
```

✓ Terminado cuando

Desde un teléfono: /start → "Compartir mi ubicación" → aparece una fila en suscriptores_telegram con su celda. Al publicar una alerta cercana, llega el mensaje con el enlace al mapa.

Paso 4.2 R2 Feed CAP para autoridades

CAP (Common Alerting Protocol) es el formato XML estándar con el que los sistemas oficiales intercambian alertas. Publicar las alertas verificadas en CAP es lo que hace a ALERTA CERCA **integrable** con Protección Civil sin rehacer nada.

```

// supabase/functions/cap/index.ts (desplegar con --no-verify-jwt: es un feed público)
import { createClient } from 'npm:@supabase/supabase-js@2';
const sb = createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!);

const esc = (s: string) => s.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
const SEVERIDAD: Record<number, string> = { 4: 'Extreme', 3: 'Severe', 2: 'Moderate', 1: 'Minor' };
const fechaCap = (iso: string) => new Date(iso).toISOString().replace(/\.d+Z$/, '+00:00');

Deno.serve(async () => {
  const { data } = await sb.from('alertas')
```

```

    .select('id, titulo, descripcion, lat, lon, radio_actual_m, verificada_en, '
      + 'categorias(nombre, nivel, categoria_cap)')
    .eq('estado', 'verificada');
const alertas = (data ?? []).map((a: any) => `
<alert xmlns="urn:oasis:names:tc:emergency:cap:1.2">
  <identifier>alertacerca-${a.id}</identifier>
  <sender>alertacerca</sender>
  <sent>${fechaCap(a.verificada_en)}</sent>
  <status>Actual</status><msgType>Alert</msgType><scope>Public</scope>
  <info>
    <language>es-MX</language>
    <category>${a.categorias.categoria_cap}</category>
    <event>${esc(a.categorias.nombre)}</event>
    <urgency>Immediate</urgency>
    <severity>${SEVERIDAD[a.categorias.nivel]}</severity>
    <certainty>Observed</certainty>
    <headline>${esc(a.titulo)}</headline>
    <description>${esc(a.descripcion ?? '')}</description>
    <area>
      <areaDesc>Radio de ${(a.radio_actual_m / 1000).toFixed(1)} km</areaDesc>
      <circle>${a.lat},${a.lon} ${(a.radio_actual_m / 1000).toFixed(1)}</circle>
    </area>
  </info>
</alert>`).join('');
return new Response(`<?xml version="1.0" encoding="UTF-8"?><n<alertas>${alertas}</n</alertas>`,
  { headers: { 'Content-Type': 'application/xml; charset=utf-8' } });
});

```

Para producción conviene publicar un feed Atom con un enlace a cada documento CAP individual, que es como lo consumen los agregadores oficiales. Para la demo basta abrir la URL en el navegador. Ejemplo completo en el Anexo B.

Paso 4.3 R1 Métricas, retención y revisión de seguridad

```

-- Métricas para el panel (security_invoker: respeta RLS, solo validadores ven todo)
create view metricas with (security_invoker = true) as
select count(*) filter (where estado in ('no_confirmada', 'corroborada', 'verificada')) as activas,
count(*) filter (where estado = 'pendiente') as por_validar,
avg(verificada_en - creada_en)
  filter (where verificada_en is not null and creada_en > now() - interval '7 days')
as tiempo_validacion,
(select count(*) from entregas where enviada_en >= current_date)
as entregas_hoy
from alertas;

-- Borrar mi cuenta (derecho de cancelación): en cascada se van perfil, dispositivos, zonas y confirmaciones
create or replace function borrar_mi_cuenta() returns void
language sql security definer set search_path = public as $$
delete from auth.users where id = auth.uid();
$$;

-- Retención: 90 días después del cierre, la alerta se anonimiza (queda solo para estadística)
select cron.schedule('anonimizar-alertas', '30 4 * * *', $$
update alertas set descripcion = null, referencia = null, foto_path = null, creada_por = null
where cerrada_en < now() - interval '90 days'
and (descripcion is not null or foto_path is not null);
$$);

```

- Los archivos de fotos se borran con la API de Storage (desde una Edge Function), no con SQL directo sobre `storage.objects`.
- Revisen *Advisors* → *Security* y *Advisors* → *Performance* en Supabase y corrijan lo que marquen.
- Prueba de seguridad P14 (Fase 5): nadie con la anon key puede leer tokens ni llamar funciones internas.

Paso 4.4 R4 Actualización en segundo plano (opcional)

Solo si la persona eligió la ubicación "todo el tiempo". Cada 15 minutos (el mínimo de Android) la app revisa su celda y la sube **solo si cambió**.

```
@pragma('vm:entry-point')
void tareaEnSegundoPlano() {
  Workmanager().executeTask((tarea, datos) async {
    await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
    await Supabase.initialize(url: supabaseUrl, anonKey: supabaseAnonKey);
    await Ubicacion.actualizarMiCelda();
    return true;
  });
}

// En main(), después de inicializar Supabase:
await Workmanager().initialize(tareaEnSegundoPlano);
await Workmanager().registerPeriodicTask('celda', 'actualizar-celda',
  frequency: const Duration(minutes: 15));
```

Paso 4.5 R5 Aviso de privacidad, diapositivas y textos legales

- **Aviso de privacidad** en la bienvenida y en Ajustes, con los puntos de la sección 12.
- **Botón "Borrar mi cuenta y mis datos"** en Ajustes (llama a borrar_mi_cuenta).
- **Leyenda fija** "ALERTA CERCA no sustituye al 911" en Inicio, en el bot y en cada mensaje de Telegram.

#	Diapositiva	Contenido
1	Portada	ALERTA CERCA, equipo, frase rectora.
2	El problema	La historia del niño en el mercado (sección 9). Hoy la información llega a quien no está cerca.
3	La idea	"Si algo importante está ocurriendo cerca de ti, deberías poder saberlo."
4	Cómo funciona	Diagrama de arquitectura simplificado (4 piezas).
5	Demo en vivo	Capturas de respaldo por si falla la conexión.
6	Radio dinámico	Diagrama de anillos.
7	Privacidad	Celdas de ~1 km y la tabla de qué se guarda y qué no.
8	Confianza	Estados, validadores, mecanismos contra el abuso y el riesgo que pocos ven.
9	¿Y sin app?	Tabla de canales y conclusión (sección 5).
10	Viabilidad	Costo \$0, piloto con empresas del CCE y escuelas, siguientes pasos.
11	Cierre	Frase rectora y QR del bot de Telegram.

8.8 Fase 5 · Pruebas, datos de la demo y ensayo (horas 34 a 40)

Paso 5.1 R5 Todos Plan de pruebas

R5 coordina; cada prueba la ejecuta alguien distinto de quien programó esa parte. Marquen cada una como aprobada o anoten el error en el tablero.

ID	Caso	Cómo se prueba	Resultado esperado
P01	Recibir sin cuenta	Instalar la app y aceptar permisos	Usuario anónimo y fila en dispositivos con celda de 6 caracteres
P02	Privacidad	Abrir la tabla dispositivos	No hay latitud ni longitud de personas, solo celdas

ID	Caso	Cómo se prueba	Resultado esperado
P03	Alerta cercana	Institución publica un incendio a 300 m de A	A la recibe en menos de 10 s con "A 300 m de ti"
P04	Alerta lejana	El mismo incendio	D (Zihuatanejo) no la recibe
P05	Radio dinámico	Menor verificado con factor_tiempo = 30	B a los ~30 s, C a los ~2 min; nadie la recibe dos veces
P06	Reporte ciudadano	Cuenta verificada reporta un asalto	Estado no_confirmada, radio $\leq$ 1 km, insignia ámbar
P07	Categoría sensible	Ciudadano reporta un menor	Estado pendiente; solo le llega al validador
P08	Duplicado	Segundo reporte igual a 200 m	No crea alerta; suma una confirmación
P09	Corroboración	Tres cuentas confirman	Pasa a corroborada; llega "AHORA CORROBORADA"
P10	Votos de falsa	Tres "parece falsa"	Regresa a pendiente (en revisión)
P11	Límite de reportes	Cuatro reportes en una hora	El cuarto se rechaza con mensaje claro
P12	Cierre	El validador la resuelve	Todos los que la recibieron ven "RESUELTA"; la foto deja de mostrarse
P13	Expiración	Forzar expira_en en el pasado	A los 5 min queda expirada y sale del mapa
P14	Seguridad	Con la anon key: leer dispositivos ajenos y llamar dispositivos_objetivo	0 filas y "permission denied"
P15	Fotos sin metadatos	Subir una foto que tenga GPS	El archivo en Storage no tiene EXIF
P16	Telegram	/start y compartir ubicación cercana	Llega el mensaje de la alerta
P17	CAP	Abrir la URL del feed	XML con la alerta verificada
P18	Token inválido	Desinstalar la app y publicar una alerta	El dispositivo queda inactivo
P19	Carga	5,000 dispositivos falsos (Paso 5.2)	Consulta espacial en milisegundos usando el índice

```
# P14: con la anon key, intentar llamar una función interna (debe fallar)
curl -X POST "https://TU_REF.supabase.co/rest/v1/rpc/dispositivos_objetivo" \
  -H "apikey: TU_ANON_KEY" -H "Content-Type: application/json" \
  -d '{"p_alerta": "00000000-0000-0000-0000-000000000000", "p_radio_m": 100000}'
# Esperado: error "permission denied for function dispositivos_objetivo"
```

Paso 5.2 R1 Prueba de carga con 5,000 teléfonos falsos

```
-- 1) 5,000 dispositivos falsos repartidos ~11 km alrededor de Lázaro Cárdenas
insert into dispositivos (usuario_id, fcm_token, plataforma, celda)
select (select id from auth.users limit 1), 'falso-' || g, 'android',
       st_geohash(st_setsrid(st_makepoint(-102.1942 + (random() - 0.5) * 0.2,
                                         17.9581 + (random() - 0.5) * 0.2), 4326), 6)
from generate_series(1, 5000) g;

-- 2) ¿Usa el índice espacial? Debe aparecer "Index Scan using dispositivos_centro_idx"
explain analyze
select d.id from alertas a, dispositivos d
where a.id = 'ID_DE_UNA_ALERTA' and d.activo and st_dwithin(d.centro_celda, a.ubicacion, 3700);

-- 3) ¡Limpiar ANTES de cualquier envío real! (si no, notificar intentará mandarles push)
delete from dispositivos where fcm_token like 'falso-%';
```

Anoten el tiempo de ejecución que reporta EXPLAIN ANALYZE: es un dato fuerte para la diapositiva de escalabilidad.

Paso 5.3 R4 R5 Preparar la demo

Teléfono	Ubicación simulada	Coordenadas	Celda	Distancia	Recibe la alerta
Suceso	Centro de Lázaro Cárdenas (aprox.)	17.9581, -102.1942	9epq4t	—	—
A	300 m al norte	17.9608, -102.1942	9epq4t	0.3 km	Al instante (1 km)
B	2.6 km al norte	17.9815, -102.1942	9epq69	2.6 km	Al llegar a 3 km (~30 s)
C	6 km al norte	18.0121, -102.1942	9epq6x	6.0 km	Al llegar a 10 km (~2 min)
D	Zihuatanejo	17.6417, -101.5517	9epu35	76.6 km	Nunca (máximo 25 km)

Distancias y celdas calculadas con el mismo algoritmo de la app. Con el margen de 700 m, B no la recibe en el primer anillo (el centro de su celda está a 2.7 km) y C no la recibe en el de 3 km (su centro está a 6.3 km).

1. **Dispositivos:** 2 teléfonos Android reales y 2 emuladores, o 4 teléfonos. Todos con la app, sesión iniciada, notificaciones permitidas, "No molestar" apagado y volumen alto.
2. **Proyección:** usen **scrcpy** (gratis y de código abierto) para mostrar las pantallas de los teléfonos en el proyector desde una laptop.
3. **Cuentas:** un validador con sesión abierta en el panel; una cuenta ciudadana verificada con número de prueba.
4. **Datos:** borren o expiren las alertas de prueba; dejen 2 o 3 alertas "de fondo" verificadas para que el mapa no se vea vacío.
5. **Foto:** usen una ilustración o una foto de un integrante adulto que dé su permiso. **Nunca** una foto real de un menor.
6. **Configuración:** factor_tiempo = 30 justo antes de la demo y de regreso a 1 al terminar.

Paso 5.4 R5 Video de respaldo y ensayo

- Graben la demo completa con OBS Studio el día anterior (pantalla del panel + pantallas de los teléfonos con scrcpy). Guárdenlo en una USB y en la nube.
- Ensayen el pitch completo al menos 3 veces con cronómetro. Cada quien debe poder cubrir la parte de otro.
- Preparen respuestas a las preguntas difíciles (sección 9.3).

9 Guion de la demostración y del pitch

9.1 La historia para abrir (45 segundos)

Para leer en voz alta

"Son las cinco de la tarde. Un niño de 8 años se separa de su mamá en el mercado. Ella pide ayuda y alguien lo publica en Facebook. En una hora, la publicación la vieron personas en Morelia, Uruapan y Guadalajara. La señora que vende jugos a 200 metros, la que vio pasar a un niño con playera roja, no se enteró. ALERTA CERCA existe para que la información le llegue primero a ella."

9.2 Minuto a minuto (7 minutos)

Tiempo	Quién	Qué se dice y qué se muestra
0:00 – 0:45	R5	La historia. El problema del reto en una frase.
0:45 – 1:30	R5	La idea y la arquitectura en una diapositiva: app, backend, panel, canales.
1:30 – 4:30	R3 R4 R5	Demo en vivo con los 4 teléfonos proyectados: (1) un ciudadano reporta "menor desaparecido" → en revisión, nadie lo recibe; (2) el validador lo verifica en el panel; (3) A suena: "a 300 m de ti"; (4) 30 s después, B; (5) 2 min después, C; D nunca; mientras tanto, el panel muestra el círculo creciendo; (6) se resuelve y todos reciben "RESUELTA".
4:30 – 5:30	R1	Privacidad y confianza: abrir la tabla dispositivos ("no hay coordenadas, solo celdas de 1 km"); estados de confianza; el riesgo de las búsquedas maliciosas.
5:30 – 6:15	R2	¿Y sin app? La tabla de canales; invitar a un juez a escanear el QR del bot de Telegram y mostrar el feed CAP.
6:15 – 7:00	R5	Viabilidad: costo \$0, piloto con empresas del CCE y escuelas, siguientes pasos. Cierre con la frase rectora.

9.3 Preguntas difíciles del jurado

Pregunta	Respuesta corta
¿Y si alguien publica una alerta falsa?	Para reportar se necesita un teléfono verificado; lo no confirmado viaja máximo 1 km; personas y menores siempre se validan; hay votos de "parece falsa", reputación y bitácora.
¿Por qué no SMS o una alerta que llegue sin app?	El único canal universal, Cell Broadcast, lo operan las autoridades con los operadores; el SMS cuesta. Nuestra salida CAP permite que una autoridad retransmita nuestras alertas verificadas por esos canales.
¿Cuánto cuesta operarlo?	El prototipo, \$0. Un piloto de miles de usuarios cabe en los planes gratuitos o en Supabase Pro (alrededor de 25 USD al mes). El único costo variable es el SMS de verificación de quienes reportan.
¿Gasta mucha batería?	No rastreamos de forma continua: usamos ubicación de precisión media solo al abrir la app, al cambiar de celda o cada 15 minutos si la persona lo permite.
¿Funciona en iPhone?	Es el mismo código Flutter. Publicarlo requiere la cuenta de Apple Developer (99 USD al año); por eso la demo es en Android.
¿Quién valida a las 3 de la mañana?	Guardias de validadores de varias instituciones. Las categorías no sensibles circulan como "no confirmadas" mientras tanto; las sensibles avisan a todos los validadores y medimos el tiempo de validación.

Pregunta	Respuesta corta
¿Y si no hay internet?	FCM entrega el mensaje al reconectarse mientras siga vigente. Para conectividad cero, el camino es Cell Broadcast vía autoridades (por eso CAP).
¿Y la privacidad del menor?	Foto solo con consentimiento del tutor y validación; al resolverse, la foto deja de mostrarse; a los 90 días la alerta se anonimiza.
¿Por qué no basta un grupo de WhatsApp?	No es geográfico, no verifica, no se cierra y no escala: es justo el problema que describe el reto.
¿Cómo evitan el pánico?	Lenguaje claro, nivel y estado visibles, distancia exacta e instrucciones ("no intervengas, llama al 911").
¿Se puede usar en otra ciudad?	Sí: nada depende de Lázaro Cárdenas. Se agregan validadores locales y una columna de municipio para separar paneles.

10 Riesgos y plan B

Riesgo	Probabilidad	Impacto	Qué hacemos
Falla el internet del evento	Media	Alto	Hotspot de dos teléfonos con datos; video de respaldo.
Las notificaciones tardan en el emulador	Media	Medio	Probar el día anterior; preferir teléfonos reales; mostrar en el panel las entregas en vivo.
El proyecto de Supabase se pausó	Media	Alto	Entrar al panel el día anterior y el mismo día.
Se suben llaves secretas a GitHub	Baja	Alto	.gitignore y revisión antes de cada push. Si pasa: rotar las llaves de inmediato.
Un integrante se atora	Alta	Medio	Juntas cada 4 horas, trabajo en pares, regla de los 30 minutos.
Cambios en las APIs de los paquetes	Media	Bajo	Fijar versiones con pubspec.lock; leer el README del paquete.
El GPS no funciona en interiores	Alta	Medio	Modo demo con ubicaciones simuladas.
Permisos denegados en plena demo	Media	Medio	Teléfonos configurados desde antes y lista de verificación (Anexo C).
Se acaba el tiempo	Alta	Alto	Tabla de prioridades (8.2) y congelar funciones 4 horas antes.

11 Escalamiento y siguientes pasos

11.1 Del prototipo al piloto

1. **Piloto de 3 meses** en colonias seleccionadas de Lázaro Cárdenas, con validadores de Protección Civil municipal, el CCE y el Tecnológico.
2. **Adopción:** empresas del CCE, escuelas y comercios con QR (sección 5.1).
3. **Medición:** tiempo de validación, tiempo hasta la primera entrega, alertas falsas detectadas y casos resueltos con ayuda de la comunidad.

11.2 Mejoras técnicas

- **iOS publicado** y, con aprobación de Apple, notificaciones críticas que suenen aunque el teléfono esté en silencio.
- **Corredores de huida** para robo de vehículos: además del círculo, avisar a lo largo de las salidas carreteras de la ciudad (por ejemplo, la autopista Siglo XXI y la carretera federal 200).

- **Varios municipios:** columna de municipio, validadores y paneles por municipio.
- **Integración CAP** con Protección Civil estatal y, a través de las autoridades, con Cell Broadcast.
- **Analítica preventiva:** mapa de calor de incidentes para el CCE y las autoridades (con datos anonimizados).

11.3 Costos estimados

Concepto	Prototipo	Piloto (miles de usuarios)	Ciudad completa
Supabase	Gratis	Gratis o Pro (~25 USD/mes)	Pro con más recursos
Firebase Cloud Messaging	Gratis	Gratis	Gratis
Bot de Telegram y mapas OSM	Gratis	Gratis*	Servidor de mapas propio o proveedor
SMS de verificación	Números de prueba	Por SMS (solo quien reporta)	Por SMS
Google Play	—	25 USD (pago único)	—
Apple Developer	—	99 USD al año	99 USD al año

* El servidor público de mapas de OpenStreetMap tiene una política de uso justo; con mucho tráfico conviene un proveedor de mosaicos o un servidor propio. Precios aproximados: verifíquenlos antes de presentarlos.

12 Privacidad y marco legal

ALERTA CERCA maneja datos personales (teléfono, ubicación aproximada) y, en algunos casos, datos sensibles (fotos de menores, condiciones de salud de personas vulnerables). Esto es lo mínimo que el prototipo debe considerar:

- **Ley aplicable:** si el sistema lo opera el CCE (un particular), aplica la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (México publicó una nueva versión en 2025; verifiquen el texto vigente). Si lo operara una institución pública, aplica la ley general para sujetos obligados.
- **Principios:** licitud, consentimiento, información, calidad, finalidad, lealtad, proporcionalidad y responsabilidad. En la práctica: pedir solo lo necesario, explicar para qué y no usarlo para otra cosa.
- **Aviso de privacidad** visible antes de pedir datos: quién es el responsable, qué datos se recaban (celda de ~1 km, token del teléfono, número de quien reporta), para qué, con quién se comparten (con nadie, salvo autoridades en alertas verificadas), cuánto tiempo se guardan y cómo ejercer los derechos.
- **Derechos ARCO** (acceso, rectificación, cancelación y oposición): botón "Borrar mi cuenta" y un correo de contacto.
- **Datos sensibles:** fotos y datos de menores o personas vulnerables solo con consentimiento expreso del tutor o familiar, únicamente en alertas validadas, y se ocultan al resolverse.
- **Conservación:** entregas, 30 días; alertas cerradas, anonimizadas a los 90 días; dispositivos inactivos, desactivados a los 60 días.
- **Seguridad:** RLS en todas las tablas, secretos en Vault, HTTPS, mínimo privilegio (las funciones internas no se pueden llamar desde la app) y bitácora de acciones.
- **Leyenda obligatoria:** "ALERTA CERCA no sustituye al 911 ni a los sistemas oficiales de alertamiento."

⚠ Ojo

Esto no es asesoría legal. Para un piloto real, el CCE debe revisar el aviso de privacidad y los términos de uso con un abogado.

A Glosario para el equipo

Término	Qué significa
Anon key / service_role key	Llaves de Supabase. La anon es pública (va en la app y respeta RLS); la service_role salta todas las reglas y solo vive en el servidor.
CAP	Common Alerting Protocol: formato XML estándar para intercambiar alertas públicas entre sistemas.
Cell Broadcast	Difusión celular: mensaje que las antenas envían a todos los teléfonos de una zona, sin app. Lo operan autoridades y operadores.
Edge Function	Función en TypeScript que corre en los servidores de Supabase y se llama por una URL.
FCM	Firestore Cloud Messaging: servicio gratuito de Google para enviar notificaciones push.
Geohash	Forma de dividir el mapa en cuadros con nombre; con 6 caracteres, cada cuadro mide $\sim 1.2 \times 0.6$ km.
MVP	Producto Mínimo Viable: la versión más pequeña que demuestra que la idea funciona.
pg_cron / pg_net	Extensiones de PostgreSQL para programar tareas periódicas y hacer peticiones HTTP desde SQL.
PostGIS	Extensión de PostgreSQL para datos geográficos (distancias, puntos, índices espaciales).
Push	Notificación que el servidor envía al teléfono aunque la app esté cerrada.
RLS	Row Level Security: reglas que filtran cada fila según quién hace la consulta.
RPC	Llamar a una función de la base de datos desde la app (<code>supabase.rpc(...)</code>).
Token FCM	Identificador único de una instalación de la app; es la "dirección" a la que se manda el push.
Validador	Persona de una institución que revisa, verifica, ajusta y cierra alertas desde el panel.
Vault	Almacén cifrado de secretos dentro de Supabase.
Webhook	URL que otro servicio (como Telegram) llama cuando ocurre algo.

B Ejemplo de una alerta en formato CAP

Ejemplo ficticio para la demo. En CAP, la hora va con desfase explícito (+00:00) y el círculo se expresa como "latitud,longitud radio" con el radio en kilómetros.

```
<?xml version="1.0" encoding="UTF-8"?>
<alert xmlns="urn:oasis:names:tc:emergency:cap:1.2">
  <identifier>alertacerca-6f1c2a9e-0b7d-4a51-9a43-2c8e7f1d5b10</identifier>
  <sender>alertacerca</sender>
  <sent>2026-11-14T23:42:00+00:00</sent>
  <status>Actual</status>
  <msgType>Alert</msgType>
  <scope>Public</scope>
  <info>
    <language>es-MX</language>
    <category>Rescue</category>
    <event>Menor desaparecido o posible sustracción</event>
    <urgency>Immediate</urgency>
    <severity>Extreme</severity>
    <certainty>Observed</certainty>
    <headline>Niño de 8 años visto por última vez frente al mercado</headline>
    <description>Playera roja y short azul, 1.20 m. Validado por Protección Civil (demo).</description>
    <area>
      <areaDesc>Radio de 3.0 km</areaDesc>
      <circle>17.9581,-102.1942 3.0</circle> <!-- Latitud,Longitud radio_en_km -->
    </area>
  </info>
</alert>
```

</alert>

C Lista de verificación para el día de la demo

Una hora antes

- ☐ Proyecto de Supabase activo (alguien ejecutó una consulta hoy).
- ☐ Tareas de pg_cron corriendo (cron.job_run_details con ejecuciones recientes).
- ☐ Alertas de prueba expiradas; 2 o 3 alertas de fondo verificadas.
- ☐ Los 4 teléfonos cargados, con la app, sesión iniciada y ubicaciones A, B, C y D en el modo demo.
- ☐ Notificaciones permitidas, volumen alto, "No molestar" apagado.
- ☐ Panel abierto con la sesión del validador.
- ☐ Cuenta ciudadana verificada lista para reportar; foto de ilustración en el teléfono.
- ☐ scrcpy probado con el proyector.
- ☐ Hotspot de respaldo encendido y probado.
- ☐ Video de respaldo en USB y en la nube.
- ☐ QR del bot de Telegram impreso o en la diapositiva final.

Justo antes de empezar

- ☐ `update config set factor_tiempo = 30;`
- ☐ Cronómetro listo; cada quien sabe cuándo le toca hablar.

Al terminar

- ☐ `update config set factor_tiempo = 1;`
- ☐ Resolver o expirar las alertas de la demo.
- ☐ Tomar capturas y anotar las preguntas del jurado para mejorar.

"Si algo importante está ocurriendo cerca de ti, deberías poder saberlo."

USAR LA TECNOLOGÍA PARA CUIDARNOS ENTRE TODOS